

A multi-view contrastive learning framework for spatial embeddings in risk modelling

Freek Holvoet^{1,2}, Christopher Blier-Wong³, and Katrien Antonio^{1,2,4}

¹*Faculty of Economics and Business, KU Leuven, Belgium.*

²*LRisk, Leuven Research Center on Insurance and Financial Risk Analysis, KU Leuven, Belgium.*

³*Department of Statistical Sciences, University of Toronto, Canada*

⁴*Faculty of Economics and Business, University of Amsterdam, The Netherlands.*

August 19, 2026

Abstract

Incorporating spatial information, particularly when related to climate, weather, and demographic factors, is crucial for improving underwriting precision and enhancing risk management in insurance. However, spatial data are often unstructured, high-dimensional, and difficult to integrate into predictive models. Embedding methods are needed to convert spatial data into meaningful representations for modelling tasks. We propose a novel multi-view contrastive learning framework for generating spatial embeddings that combine information from multiple spatial data sources. To train the model, we construct a spatial dataset that merges satellite imagery and OpenStreetMap features across Europe. The framework aligns these spatial views with coordinate-based encodings, producing low-dimensional embeddings that capture both spatial structure and contextual similarity. Once trained, the model generates embeddings directly from latitude–longitude pairs, enabling any dataset with coordinates to be enriched with meaningful spatial features without requiring access to the original spatial inputs. In a case study on French real estate prices, we compare models trained on raw coordinates against those using our spatial embeddings as inputs. The embeddings consistently improve predictive accuracy across generalised linear, additive, and boosting models, while providing post-hoc explainable spatial effects and demonstrating generalisation of the fitted spatial effects to regions without training observations. A second case study on flood claim counts across Belgian postal codes confirms that the embeddings improve territorial risk classification in an insurance context.

Keywords: spatial embedding, multi-view learning, GeoAI, spatial representation learning, risk analysis, satellite imagery

1 Introduction

Spatial data are central to fields such as environmental monitoring, urban planning, and real estate pricing. Spatial modelling is also increasingly important in insurance applications. For instance, spatial data are used to link weather patterns to hail-damage claim frequency (Gao

and Shi 2022) or to quantify the effect of heat waves on mortality rates (Robben et al. 2026). These examples illustrate how incorporating spatial information enables actuaries to better analyse and manage risks that exhibit spatial dependence. Despite their importance, integrating spatial information into predictive models remains challenging. Location is often represented as a simple coordinate pair (latitude and longitude) or as a categorical variable, which fails to capture real-world geographical structure. Popular machine learning methods, such as random forests and gradient boosting machines, apply linear splits on these coordinate variables, leading to artificial rectangular patterns that do not align with real-world geography (Geerts et al. 2024).

Early work in spatial risk modelling was primarily inspired by geostatistics, where the focus is on understanding the dependence structures inherent in spatially distributed data. Two modelling strategies are typically explored: incorporating location via spatial smoothing of residuals, see Taylor (1989); Boskov and Verrall (1994); Taylor (2001) or by considering the spatial coordinates directly in a statistical model, for example, in a generalised additive model with a bivariate smooth effect of latitude and longitude, see Dimakos and Di Rattalma (2002); Gschlößl and Czado (2007); Shi and Shi (2017); Henckaerts et al. (2018, 2021). While these approaches model spatial dependence explicitly, modern machine learning increasingly represents spatial information through representation learning, where spatial information is captured implicitly via so-called embeddings.

Embeddings are vector representations of data objects, obtained through an encoder that maps discrete or structured inputs into a continuous, typically low-dimensional space. The dimensionality of the embedding determines how much information it can represent: low-dimensional embeddings capture the most essential structure of the input data, while higher-dimensional embeddings can preserve more detailed or complex patterns. A particular class of embeddings, called entity embeddings, refers to compact, continuous-valued representations of categorical variables, where each category is mapped to a low-dimensional vector (Guo and Berkhahn 2016). The use of embeddings in actuarial science is not new: Richman (2021) discusses supervised entity embeddings for categorical variables, and Blier-Wong et al. (2021) construct entity embeddings via unsupervised learning. In a motor third-party liability dataset, categorical variables such as gender, vehicle type, and coverage level can each be represented by entity embeddings. Applications in non-life insurance pricing include Delong and Kozak (2023); Avanzi et al. (2024); Richman and Wüthrich (2024); Wilsens et al. (2025); Holvoet et al. (2025). Spatial information can be categorical in nature, for instance, when postal codes or administrative regions are treated as levels of a categorical variable. In such cases, entity embeddings can be used to capture spatial variation in risk levels, as shown in Shi and Shi (2023). In Blier-Wong et al. (2022), the authors claim that entity embeddings are inappropriate for integrating spatial information in insurance loss modelling. Granular territories such as postal codes are typically created to facilitate mail delivery, not to capture homogeneous risk profiles. Furthermore, if neighbours live on the border between two territories, their insurance premiums may differ since the embeddings of their territories will vary, even though their spatial risk should be similar. For this reason, the effect of spatial information in prediction models should be smooth on a map, and entity embeddings typically do not possess this property.

For these reasons, Blier-Wong et al. (2022) propose modelling spatial risk through spatial embeddings rather than entity embeddings. Spatial embeddings represent geographic information directly in a continuous latent space, where locations that are close in space are assigned similar vectors. Unlike entity embeddings, which treat each spatial unit as an independent category, spatial embeddings encode smooth transitions across space.

Spatial embeddings can also incorporate external contextual information about each location, such

as census statistics, land cover maps, or satellite imagery. These context-aware spatial embeddings enable models to represent not only the location itself, but also what is present there. In a case study on claim frequency modelling using Canadian data, [Blier-Wong et al. \(2022\)](#) construct spatial embeddings from geographic coordinates enriched with census data using a convolutional neural network. They show that these embeddings capture spatial risk patterns more effectively than traditional spatial smoothing methods, such as generalised additive models with bivariate smooth effects. Similarly, [Moussaid \(2023\)](#) constructs spatial embeddings based on H3 hexagonal coordinates ([Uber Technologies, Inc. n.d.](#)), augmenting them with spatial context from OpenStreetMap data for each hexagonal cell. Using a Belgian motor third-party liability claims dataset, they demonstrate that incorporating such contextual information improves predictive accuracy. These studies provide direct evidence that context-aware spatial embeddings are an effective tool for actuarial risk modelling, with [Blier-Wong et al. \(2022\)](#) demonstrating their value on home insurance pricing for a Canadian portfolio. The present paper builds on this evidence and focuses on the construction of the embedding model itself, by proposing a framework that combines multiple spatial data sources within a single contrastive learning architecture.

In the broader field of geospatial artificial intelligence (GeoAI), spatial representation learning has advanced rapidly through neural networks that process large-scale, heterogeneous spatial data ([Yin et al. 2019](#); [Blier-Wong et al. 2020](#); [Mai et al. 2020, 2022](#)). These methods typically produce task-agnostic spatial embeddings, which means that the embeddings are not trained for a single predictive task, but rather to capture general spatial structure and relationships across different regions or data sources. Once learnt, these embeddings can serve as general-purpose spatial representations that can be reused in different predictive settings. This property makes them particularly suited for the generalisation of fitted spatial effects to territories where no outcome observations are available. By leveraging shared geographic characteristics through spatial embeddings, a downstream model can produce meaningful predictions in locations without training data.

The recent development of foundation models extends context-aware spatial embedding models by aiming to create general-purpose geospatial representations that integrate multiple data modalities while retaining spatial awareness. Foundation models have already transformed language and vision through large pretrained architectures, and similar ambitions are now emerging for multimodal spatial models that unify imagery, vector data, and semantic context ([Mai et al. 2024, 2025](#)). A recent example is AlphaEarth ([Brown et al. 2025](#)), a large-scale multi-view and spatiotemporal geospatial foundation model that integrates satellite imagery, weather reanalysis, and auxiliary contextual data to produce globally consistent embeddings. One specific spatial foundational model architecture we will investigate in this paper is the SatCLIP model, proposed by [Klemmer et al. \(2023, 2025\)](#). SatCLIP constructs context-aware spatial embeddings by combining geographic coordinates with visual context extracted from satellite imagery. The model produces two types of embeddings: one derived from spatial coordinates using spherical harmonics to capture global spatial structure ([Rußwurm et al. 2024](#)), and another derived from satellite images using a vision transformer that extracts high-level visual patterns ([Tarasiou et al. 2023](#)). These two embedding spaces are aligned through contrastive learning, where the model learns to assign similar representations to spatial locations that share similar visual and geographic characteristics, and dissimilar representations to locations that differ. The contrastive training procedure allows the coordinate encoder to internalise the contextual information contained in the imagery. As a result, SatCLIP can generate embeddings for new locations using only their geographic coordinates, without requiring access to the original spatial data. For this reason we build on the SatCLIP architecture. In actuarial

applications a dataset typically records the location of a policyholder or insured object but no further spatial context, so useful and rich embeddings must be generated from coordinates alone. Alternative context-aware approaches in the actuarial literature, such as [Blier-Wong et al. \(2022\)](#) and [Moussaid \(2023\)](#), construct embeddings directly from the contextual data and therefore require access to these data sources whenever embeddings are produced for new locations. [Klemmer et al. \(2025\)](#) uses the trained SatCLIP model to capture large-scale spatial variation across the entire earth’s surface resulting in a high-dimensional embedding. They demonstrate the use of these embeddings in a case study on global temperature pattern prediction.

In this paper, we propose a multi-view contrastive learning model that generates spatial embeddings by leveraging multiple input data sources as spatial context. Our contribution lies in combining information from different spatial views with geographical coordinates to obtain low-dimensional embeddings that capture local spatial patterns. This contribution is methodological in nature and applies to spatial representation learning in general. The design choices, however, are guided by actuarial practice, where datasets record the location of policyholders or insured objects and spatial features must be available from coordinates alone. To the best of our knowledge, an architecture that fuses multiple spatial views with a coordinate encoder through contrastive learning has not yet been proposed in the actuarial literature. The model takes as input different sources of spatial data, such as satellite imagery and tabular neighbourhood information, together with the geographical coordinates of where the spatial information was gathered. Through contrastive learning, the model learns to associate patterns in the geographic coordinates with the information contained in the spatial views. Rather than targeting global coverage, the proposed framework is intended for regional settings. It can be retrained efficiently on consumer-grade hardware, flexibly incorporates different spatial data sources, and yields fine-grained spatial embeddings that capture local geographic structure in greater detail when compared to global-scale spatial embedding models. Our work differs from SatCLIP ([Klemmer et al. 2023, 2025](#)) in two key aspects. First, we integrate multiple spatial data sources rather than relying on a single modality, such as satellite imagery. Second, our model is designed to learn compact embeddings optimised for fine-grained regional applications, whereas SatCLIP produces high-dimensional global representations tailored to large-scale earth system modelling. Figure 1 gives a schematic overview of the framework, with all details deferred to Section 3. The model consists of two parts. The first part, shown at the bottom of the figure, summarises the available spatial data for a location, in our case a satellite image and neighbourhood information from OpenStreetMap, into a single vector. The blue blocks encode each spatial view and the orange block fuses these into one vector. The second part, shown at the top, converts the latitude and longitude coordinates of that location into a vector of the same dimension, represented by the green block. During training, the model compares the two vectors and adjusts both parts so that, for each location, the coordinate-based vector matches the data-based vector, while differing from the vectors of other locations. After training, the green block on its own produces the spatial embedding. The embedding of any new location can therefore be computed from its coordinates, without requiring access to the underlying spatial data.

This paper is structured as follows. Section 2 details the construction of a multi-view spatial dataset for European countries. We gather both satellite images via Google Maps ([Google n.d.](#)) and neighbourhood context via OpenStreetMap ([OpenStreetMap contributors n.d.](#)) for almost 100 000 locations across Europe. Section 3 shows the construction of the multi-view spatial embedding model and details the contrastive loss function, the training, and the hyperparameter choices. In Section 4, we illustrate the practical value of our spatial embeddings in two case studies. The case

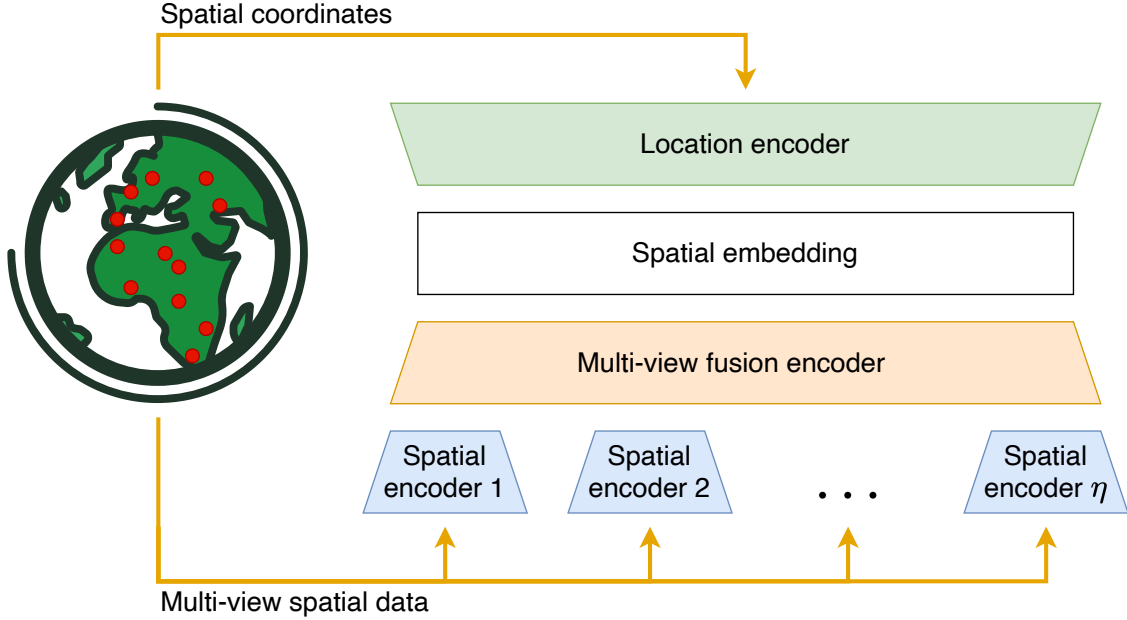


Figure 1: Schematic overview of our multi-view spatial embedding model. At the bottom, each blue block encodes one spatial view, and the orange block fuses these into a single spatial view embedding. At the top, the green block encodes the latitude and longitude coordinates. The model is trained with a contrastive learning objective, which aligns the coordinate embedding of a location with the embedding of its spatial views. Image inspired by [Klemmer et al. \(2025\)](#).

studies serve as a demonstration and validation of this methodology, rather than as fully developed pricing applications. The first case study considers the context of real estate pricing. Traditional models in this field often rely on basic location indicators, such as postal codes or proximity to landmarks, which only provide limited spatial context. In contrast, our proposed spatial embeddings incorporate a richer set of geographic features, enabling the model to better account for the influence of surrounding areas and environmental characteristics on property values. To evaluate our approach, we compare the predictive performance for predicting real estate prices using our spatial embeddings versus raw coordinates. We use permutation-based variable importance and partial dependence effects to examine how the embeddings influence the predictions, and evaluate how the fitted spatial effects generalise to areas held out of training. The second case study, in Section 4.2, applies the embeddings in an insurance setting and models flood claim counts across Belgian postal codes. Section 5 concludes the paper.

2 Dataset of locations and spatial views

We construct a multi-view dataset of 95 857 unique locations distributed across the European continent. The sampling of locations is guided by population distribution, using the European Local Administrative Units (LAU) framework ([Eurostat 2021](#)). The location encoder learns spatial variation in more detail where the training data contains more sampled locations. Sampling proportionally to population therefore places this detail where most insured objects are located. The urban or rural character of a location is captured by the spatial views, so the sampling only

determines how finely the location encoder can distinguish nearby locations, not what information the embeddings contain. For case studies that require finer detail in rural areas, such as agricultural insurance, the sampling scheme can be adapted to place more training locations in those regions. We show the distribution of the dataset’s locations in Europe in Figure 2.

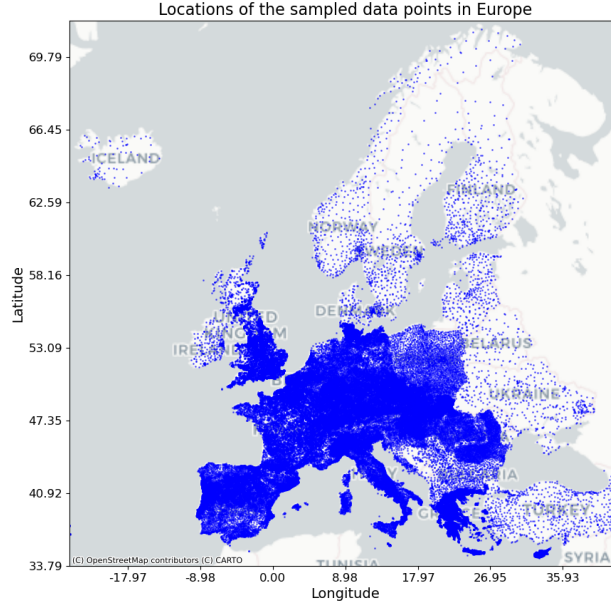


Figure 2: Our dataset consists of 95 857 sampled locations in European countries. The sampling of the locations is done based on the European Local Administrative Units framework.

Each location is defined by a latitude–longitude coordinate pair and enriched with two data views: a Google satellite image (GS) on the one hand and structured semantic context from OpenStreetMap (OSM) ([OpenStreetMap contributors n.d.](#)) on the other hand. Figure 3 shows an example of the two spatial views for a location in Leuven, Belgium. We gather the satellite image using the Google Maps Static API ([Google n.d.](#)), as shown in Figure 3b. The satellite image is centred on the coordinates of the location and has a resolution of 256×256 pixels in RGB format, resulting in a dimension of $256 \times 256 \times 3$. We gather OSM tags representing semantic information typically not visible in satellite imagery. Hereto, we define six categories of tags: *shops*, *tourism*, *food & drink*, *health*, *education* and *public safety*. Appendix A contains all OpenStreetMap tags corresponding to each category. We aggregate occurrences of these tags in three rings of H3 hexagonal cells ([Uber Technologies, Inc. n.d.](#)) centred at each location. Using concentric rings around the centre, we capture both close-by and more distant amenities in a consistent spatial structure. This is illustrated in Figure 3c, which shows three rings of hexagons around the centre location with the occurrences of each category of tag in coloured points. Three rings of hexagons around a central location lead to 37 hexagons in total, where the frequency of each of the six tag categories is counted for each hexagon.

Formally, let $P = 95\,857$ denote the total number of locations in our dataset. We define the dataset $\mathcal{D} = \{(\lambda_p, \phi_p), V_p^{\text{GS}}, V_p^{\text{OSM}}\}_{p=1}^P$, where the pair (λ_p, ϕ_p) represents the geographical coordinates latitude and longitude of location p . Each location is associated with a satellite image represented by $V_p^{\text{GS}} \in \mathbb{R}^{256 \times 256 \times 3}$, and $V_p^{\text{OSM}} \in \mathbb{N}^{37 \times 6}$ represents the aggregated OpenStreetMap tag counts.

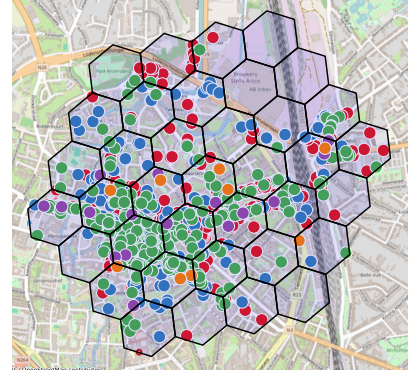
Note that for a location p , the spatial views V_p^{GS} and V_p^{OSM} always correspond to the coordinates (λ_p, ϕ_p) .



(a) The coordinate pair $(\lambda, \phi) = (50.88173^\circ, 4.70591^\circ)$ shown with a red cross. This corresponds to a location in Leuven, Belgium.



(b) A Google Satellite image corresponding to the coordinate pair from panel (a). The data point V^{GS} represents the RGB pixel values from this image.



(c) OpenStreetMap tags corresponding to the coordinate pair from panel (a). The data point V^{OSM} contains the count for each of the six categories in each of the hexagons.

Figure 3: Illustration of one data point in the dataset $\mathcal{D}$, with coordinates $(\lambda, \phi) = (50.88173^\circ, 4.70591^\circ)$ in Leuven, Belgium. Panel (a) shows the location of the coordinate pair. Panel (b) displays the associated Google Satellite image, where the matrix $V^{\text{GS}} \in \mathbb{R}^{256 \times 256 \times 3}$ stores the associated RGB pixel values. Panel (c) displays the corresponding OpenStreetMap features, where the matrix $V^{\text{OSM}} \in \mathbb{N}^{37 \times 6}$ contains the counts of six categories of tags for each of the 37 surrounding hexagons.

3 Spatial embedding model architecture

Using the two spatial views introduced in Section 2, we construct our spatial embedding model. Firstly, we introduce the two spatial view encoders and the structure of the multi-view fusion encoder as sketched in Figure 1. Secondly, we detail the architecture of the location encoder. Thirdly, we provide an overview of the model’s complete architecture, training, loss function, and examine the hyperparameter choices for training the spatial embedding model.

3.1 Spatial view encoders and multi-view fusion

Google satellite encoder (GS) The image encoder, denoted as $f^{\text{GS}}(V^{\text{GS}})$, maps a satellite image $V^{\text{GS}} \in \mathbb{R}^{256 \times 256 \times 3}$ to a d_{GS} -dimensional vector. As encoder f^{GS} , we use a convolutional neural network, since such architectures are specifically designed to extract and combine patterns from image data. In particular, we adopt a ResNet18 model (He et al. 2016), a convolutional neural network, trained on satellite imagery using momentum contrast (He et al. 2020). Momentum contrast is a self-supervised method that trains a network to represent similar inputs by similar vectors. The network is pretrained on SSL4EO-S12 (Wang et al. 2023), a large-scale collection of satellite imagery assembled for self-supervised learning. Wang et al. (2023) report strong performance of the pretrained network on downstream remote sensing tasks. Starting from a pretrained

network avoids training an image encoder from scratch, which would require far more data and computation. Among the published pretrained encoders, ResNet18 is the smallest architecture, which matches our aim of a framework that can be retrained on consumer-grade hardware. The ResNet18 architecture consists of 17 convolutional layers with residual links to facilitate training (He et al. 2016). Each convolutional layer consists of several convolutional filters, meant to extract patterns from the input data. The number and size of these filters are defined by the ResNet18 architecture (Wang et al. 2023) and are not tuned since we adopt the pretrained network directly. The last convolutional layer is connected to a pooling layer and flattened into a vector, which is then passed through a fully connected network to produce the final embedding. Figure 4 illustrates this architecture. The input satellite image V^{GS} is represented by a green input block. It is processed through a sequence of convolutional layers (purple blocks), followed by pooling and flattening operations (yellow blocks). The resulting representation is then passed through a fully connected network (rightmost purple block), which outputs the final embedding vector of dimension d_{GS} shown in red.

We initialise the model with pretrained weights from Wang et al. (2023) and adjust the output dimension to d_{GS} , a hyperparameter. The trainable parameters of f^{GS} lie in the convolutional filters and in the fully connected output network, represented as purple blocks in Figure 4.

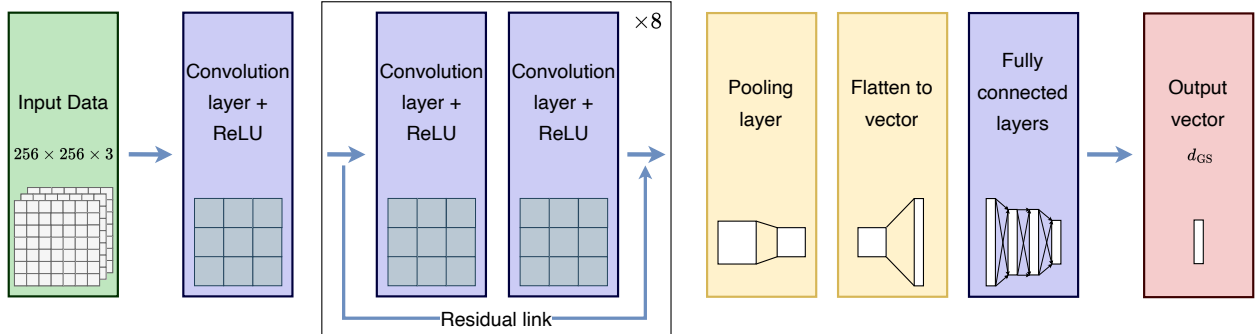
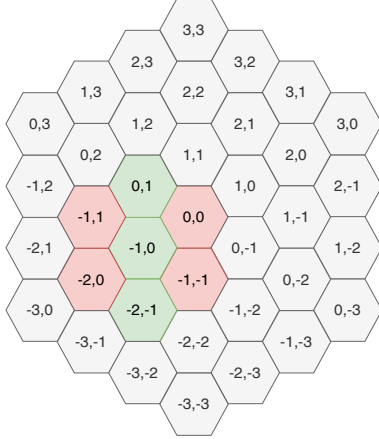


Figure 4: Illustration of the encoder used for the Google satellite imagery spatial view. We use a ResNet18 architecture (He et al. 2016), pretrained following Wang et al. (2023). The input image (green block) of dimension $256 \times 256 \times 3$ is processed by convolutional layers (purple blocks) that extract spatial patterns. These patterns are then pooled and flattened (yellow blocks) before passing through a fully connected network (rightmost purple block). The final embedding is a vector of dimension d_{GS} , as shown in the red block. Purple blocks contain the trainable parameters of the model.

OpenStreetMap tags encoder (OSM) We denote the encoder for the OpenStreetMap tags view as $f^{\text{OSM}}(V^{\text{OSM}})$ with the encoder mapping a hexagonal data input $V^{\text{OSM}} \in \mathbb{N}^{37 \times 6}$ to a d_{OSM} -dimensional vector. Similar to the encoder f^{GS} , we use a convolutional neural network to extract patterns from the input data and represent those patterns in a vector. However, our input is defined on a hexagonal grid, whereas standard convolutional neural networks are designed for square-grid data such as images. Therefore, we adopt the mapping proposed by Moussaid (2023) for convolutional neural networks on hexagonal input data while preserving neighbourhood relations between cells. The mapping follows the h2 coordinate system (Snyder et al. 1999), which minimises distortion by maintaining adjacency and approximate distance between neighbouring hexagons. Although

alternative mappings are possible, this approach provides a consistent alignment between hexagonal neighbourhoods and square convolutional filters, allowing the use of standard convolutional layers without altering the underlying spatial structure of the data. Alternative strategies exist to handle hexagonally structured inputs. The hexagonal cells can be shifted into a rectangular array combined with adapted convolution operations, as implemented in HexagDLy (Steppa and Holch 2019). Convolutions that operate natively on the hexagonal lattice are possible as well (Hoogeboom et al. 2018). Both strategies require specialised convolution implementations.

We illustrate this transformation with a grid of 37 hexagons in Figure 5a, which is mapped to a 7×7 matrix in Figure 5b following the h2 coordinate system (Snyder et al. 1999), which assigns two-axis coordinates to the cells of a hexagonal grid. The upper and lower corners of the matrix are masked, meaning that these entries are fixed to zero because no corresponding hexagons exist in those positions. For clarity, coordinates are added to each hexagonal cell and their correspondence to the square matrix is shown. A convolutional filter covering one ring of hexagons is illustrated with red and green cells in Figure 5a. In the transformed representation, this filter corresponds to a 3×3 convolutional filter, where the upper-right and lower-left corners are masked, shown with the red and green cells in Figure 5b. Generally, for a convolutional filter of k -rings around a central hexagon, the corresponding standard convolutional filter has a size of $2k + 1$, with zeros masking the upper-right and lower-left corners of size k . Hence, the encoder f^{OSM} is built using a standard convolutional architecture on transformed input data and by fixing the upper right and lower left corners of the convolutional filters to zero.



(a) A hexagonal grid of 37 cells (three rings). Coordinates are added for clarity. In colour, a convolutional filter of size $k = 1$ (one ring) is shown.



(b) The corresponding 7×7 square grid obtained from the transformation of the 37 hexagonal cells following Moussaid (2023). Coordinates indicate how hexagonal cells map to the square representation. In colour, the $k = 1$ filter is shown after transformation, corresponding to a 3×3 filter with the upper-right and lower-left corners masked.

Figure 5: Illustration of the transformation from hexagonal to square representation. A convolutional filter of size $k = 1$ on the hexagonal grid (left) corresponds to a 3×3 filter on the transformed square grid (right), with the upper-right and lower-left corners masked.

The encoder f^{OSM} is illustrated in Figure 6. The input is a hexagonal grid consisting of 37 cells, where each cell counts the number of tags for each of the six OSM categories under consideration,

resulting in an input dimension of 37×6 . We apply the transformation on our hexagonal input, which results in a square matrix of dimension $7 \times 7 \times 6$. On the transformed input, we apply two convolutional layers. Two layers suffice to cover the full spatial extent of the small 7×7 input while keeping the number of trainable parameters limited. The number of convolutional filters in each layer, together with the size of the filters, is treated as a tuning parameter, selected during the preliminary testing described in Appendix B and guided by training stability and the representational quality of the resulting embeddings rather than formal optimisation. Each convolutional layer is followed by normalisation and ReLU activation to facilitate training. The output of the second convolutional layer is then flattened into a vector and passed through a feed-forward neural network to produce the final representation of dimension d_{OSM} . This architecture enables the encoder to learn from the spatial structure of the OSM data, resulting in an output vector that accurately represents the spatial patterns in the input data.

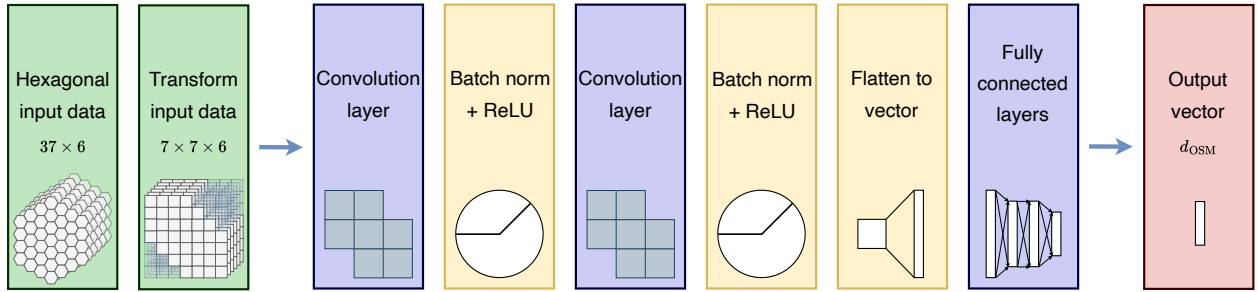


Figure 6: Illustration of the encoder used for the OpenStreetMap spatial view. The hexagonal input data is first transformed into a rectangular format. Next, there are two convolutional layers, each using masked convolutions. Finally, a feed-forward neural network outputs the encoded input. Each purple block in the flowchart contains trainable elements to be optimised in the training of the spatial encoder model.

The encoder f^{OSM} has trainable parameters in the filters of each of the two convolutional layers and in the fully connected network that produces the output. Figure 6 shows these components as purple blocks. The output dimension d_{OSM} is a hyperparameter.

Multi-view fusion We combine the embedding vectors produced by the spatial view encoders f^{GS} and f^{OSM} into a single joint spatial representation by use of a multi-view fusion encoder f^{MV} . The two embeddings are concatenated and passed through a feed-forward neural network with fully connected layers, which outputs a vector of dimension d . These layers contain trainable weights that are optimised jointly with the rest of the spatial embedding model. The depth of the network and the width of each layer are treated as hyperparameters in the model design.

3.2 Location encoder

Each data point’s coordinates (λ, ϕ) are embedded onto a d -dimensional vector by a location encoder, as illustrated in Figure 7. We denote the location encoder as

$$f^{\text{LE}} : \mathcal{D} \rightarrow \mathbb{R}^d : (\lambda, \phi) \mapsto f^{\text{LE}}(\lambda, \phi). \quad (1)$$

We use the proposed location encoder by Rußwurm et al. (2024), where $f^{\text{LE}}(\lambda, \phi) = \text{NN}(\text{PE}(\lambda, \phi))$, consisting of a positional encoder $\text{PE}(\lambda, \phi)$ and a neural network part $\text{NN}(\cdot)$. Rußwurm et al. (2024) apply this setup to learn spatial patterns from weather variables, i.e., temperature and air pressure data, for use in a regression model, and from satellite imagery for land-use classification and land-ocean segmentation.

The positional encoder is defined as

$$\text{PE}(\lambda, \phi) = [w_\ell^m Y_\ell^m(\lambda, \phi)]_{\ell=0, \dots, L; m=-\ell, \dots, \ell}, \quad (2)$$

where w_ℓ^m are weights and Y_ℓ^m are the orthogonal spherical harmonic basis functions associated with Legendre polynomials P_ℓ^m of degree ℓ and order m , see Rußwurm et al. (2024). These spherical harmonic functions encode spatial locations on a spherical surface by decomposing them into a sum of orthogonal components. The orthogonality ensures that different spatial components are captured separately, allowing the model to represent complex spatial patterns at global and localised scales. The neural network part, known as the Siren network (Sitzmann et al. 2020; Klemmer et al. 2025; Rußwurm et al. 2024), takes as input the weighted spherical harmonic functions $w_\ell^m Y_\ell^m(\lambda, \phi)$. These are passed through feed-forward layers with sinusoidal activation functions, which allow the network to represent spatial patterns smoothly on a sphere.

Figure 7 sketches the architecture of the location encoder. The input is a coordinate pair (λ, ϕ) , represented by the green block. We evaluate the spherical harmonic basis functions (yellow blocks), for $\ell = 0, \dots, L$ and for each ℓ over $m = -\ell, \dots, \ell$, using this coordinate pair as input. The weighted spherical harmonic basis functions (first purple block) serve as inputs to the feed-forward neural network with sinusoidal activation functions, known as the Siren network (second purple block). The output of the Siren network is the location embedding of the coordinate pair.

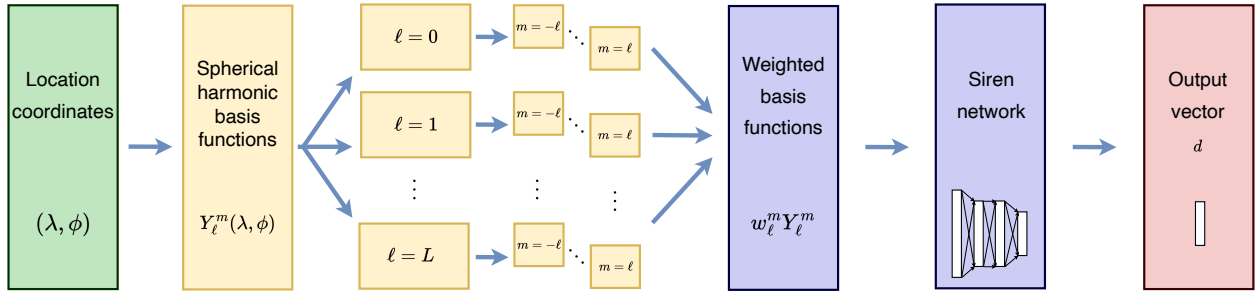


Figure 7: Illustration of the used location encoder. The weighted spherical harmonic basis functions $w_\ell^m Y_\ell^m$ are inputs to a feed-forward neural network using sinusoidal activation functions, called the Siren net (Rußwurm et al. 2024). Each purple block in the flowchart contains trainable elements that can be optimised during the training of the spatial encoder model.

The weights of the spherical harmonic basis functions and the weights in the Siren network are trainable parameters of the location encoder. Increasing the number of basis functions L allows the positional encoder to capture more fine-grained spatial detail, but also increases computational cost. To effectively utilise this additional detail, a larger value for L typically implies a larger value for d , ensuring the encoder has sufficient capacity to represent the information (Klemmer et al. 2025). The values of L , d , and the architecture of the Siren network are treated as hyperparameters.

3.3 Putting it all together: model architecture, training, and applications

Figure 8 illustrates the complete architecture for our spatial embedding model using two spatial views as inputs. The left-hand side represents the encoding of the location coordinates (λ, ϕ) by the location encoder f^{LE} as discussed in Section 3.2, resulting in an embedded location vector of dimension d . On the right, the spatial view encoders, as discussed in Section 3.1, process the two input data sources: the OpenStreetMap tags, encoded by $f^{\text{OSM}}(V^{\text{OSM}})$, into a vector of dimension d_{OSM} , and the Google satellite imagery, encoded by $f^{\text{GS}}(V^{\text{GS}})$, into a vector of dimension d_{GS} . These two spatial view embeddings are then combined by the multi-view fusion encoder $f^{\text{MV}}(f^{\text{GS}}(V^{\text{GS}}), f^{\text{OSM}}(V^{\text{OSM}}))$ into a single joint embedding of dimension d , with typically $d \leq d_{\text{GS}} + d_{\text{OSM}}$.

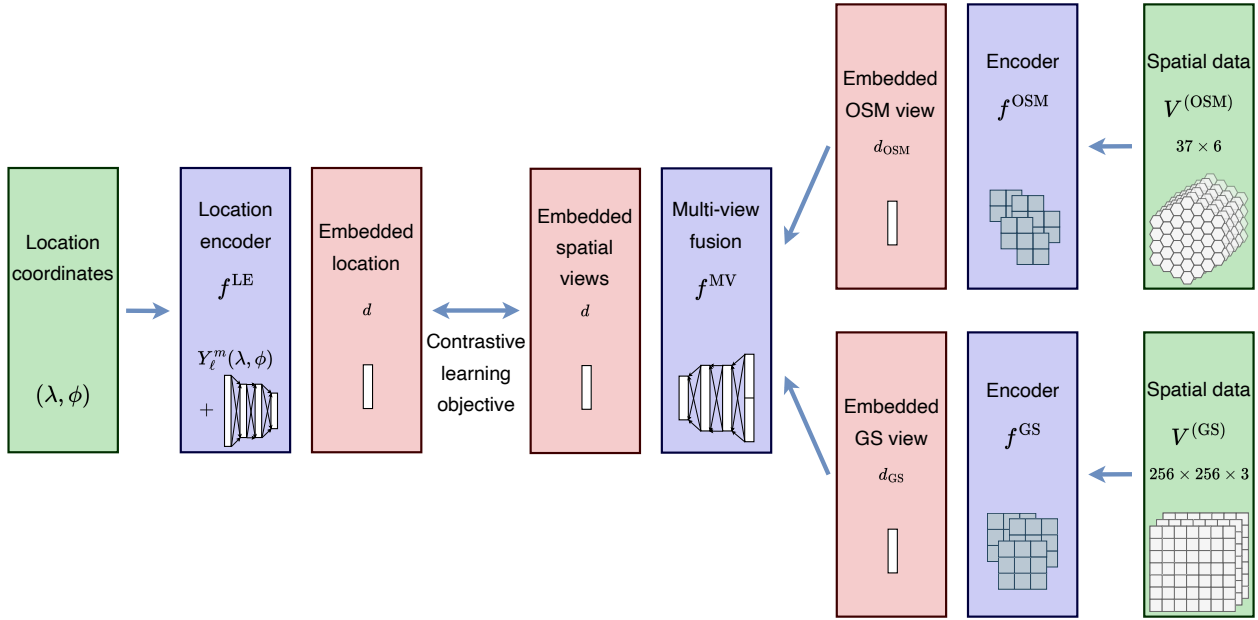


Figure 8: Architecture of the multi-view spatial embedding model with two spatial views as inputs. The left side shows the location encoder f^{LE} (Figure 7), which embeds the coordinates (λ, ϕ) . On the right, the Google satellite imagery and OpenStreetMap tags are processed by their respective encoders f^{GS} and f^{OSM} (Figures 4 and 6). Their outputs are combined in the multi-view fusion encoder f^{MV} . The resulting spatial view embedding is aligned with the location embedding using contrastive learning. Purple blocks indicate trainable components that are optimised during model training.

From the dataset $\mathcal{D}$, introduced in Section 2, we construct a training dataset $\mathcal{D}^T = \{(\lambda_p, \phi_p), V_p^{\text{GS}}, V_p^{\text{OSM}}\}_{p=1}^{P^T}$ of size P^T , containing 90% of the dataset $\mathcal{D}$ sampled randomly. The remaining 10% of data forms the validation set $\mathcal{D}^V$ and is used for early stopping in the training of the model. The training objective is to align the embedded locations with the embedded spatial views. For each $p = 1, \dots, P^T$, we denote the embedding vectors as z for notational clarity:

$$z_p^{\text{LE}} = f^{\text{LE}}(\lambda_p, \phi_p), \quad z_p^{\text{MV}} = f^{\text{MV}}(f^{\text{GS}}(V_p^{\text{GS}}), f^{\text{OSM}}(V_p^{\text{OSM}})).$$

The contrastive loss function $\mathcal{L}$ is constructed to maximise the similarity between the so called positive pairs z_p^{LE} and z_p^{MV} , and to minimise the similarity across all mismatched pairs z_p^{LE} and

z_p^{MV} for $p' \neq p$, see Radford et al. (2021). The contrastive loss over all elements in the training dataset is defined as

$$\mathcal{L} = \frac{1}{2P^T} \sum_{p=1}^{P^T} \left[-\log \frac{\exp(\text{sim}(z_p^{\text{LE}}, z_p^{\text{MV}})/\tau)}{\sum_{p'=1}^{P^T} \exp(\text{sim}(z_p^{\text{LE}}, z_{p'}^{\text{MV}})/\tau)} - \log \frac{\exp(\text{sim}(z_p^{\text{MV}}, z_p^{\text{LE}})/\tau)}{\sum_{p'=1}^{P^T} \exp(\text{sim}(z_p^{\text{MV}}, z_{p'}^{\text{LE}})/\tau)} \right],$$

where $\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$ denotes the normalised dot-product and τ is a so-called temperature hyperparameter. The two similarity terms ensure that both the alignment from location to spatial embeddings, $\text{sim}(z_p^{\text{LE}}, z_p^{\text{MV}})$, and from spatial to location embeddings, $\text{sim}(z_p^{\text{MV}}, z_p^{\text{LE}})$, is maximised. Training the complete architecture of our spatially embedded model consists of finding the optimal values for the trainable parameters so that the contrastive loss $\mathcal{L}$ on the training dataset $\mathcal{D}^T$ is minimised. The trainable parameters in the model architecture are illustrated with purple blocks in Figure 8. Minimising the contrastive loss encourages the location embeddings z_p^{LE} and the multi-view embeddings z_p^{MV} of the same point p to be similar, while embeddings of different locations remain distinct. In this way, the positional patterns represented by the spherical harmonics are aligned with the information extracted from the spatial views. The resulting embedded locations thus encode both the geographic position and the spatial content, allowing the model to represent locations in a manner that reflects where they are as well as what is observed there.

In a case study involving geographical locations, the trained spatial embedding model can be applied to represent each location by its embedding. For a dataset consisting of covariates $x_1, \dots, x_p$, where x_1 represents geographical coordinates, the covariate x_1 can be replaced with its embedding $f^{\text{LE}}(x_1)$, as illustrated in Figure 9. These embeddings act as compact descriptors that integrate both geographic position and the spatial information captured from multiple data views, and can be used as input features in downstream analyses such as regression models.

3.4 Hyperparameter choices and trained model architectures

In the construction and training of our spatial embedding model architecture, several hyperparameters have to be specified. The OpenStreetMap encoder f^{OSM} consists of two convolutional layers, the first with 8 filters and the second with 16 filters, both with kernel size $k = 1$. These values were selected during preliminary testing, as detailed in Appendix B. The multi-view fusion encoder is implemented as a single dense layer consisting of 128 neurons. The location encoder uses $L = 64$ spherical harmonic basis functions in the positional encoder, followed by a two-layer feed-forward network with 128 neurons each. Optimisation is performed with the Adam algorithm (Kingma and Ba 2015) using a learning rate of 10^{-5} and weight decay of 0.01. Training is carried out for 500 epochs with a batch size of 128. The contrastive loss employs a temperature parameter of $\tau = 0.07$. The hyperparameter choices either follow the values proposed in Klemmer et al. (2025) or were determined during preliminary testing, guided by training stability and the representational quality of the resulting embeddings rather than formal optimisation, in the model construction phase. More details can be found in Appendix B.

For the experimental study in Section 4, we study the effect of the overall embedding dimension d , and the intermediate dimensions d_{GS} and d_{OSM} of the Google Satellite and OpenStreetMap encoders on the resulting spatial embeddings. To this end, we construct five different embedding models, each corresponding to a different combination of d , d_{GS} , and d_{OSM} . Table 1 shows the

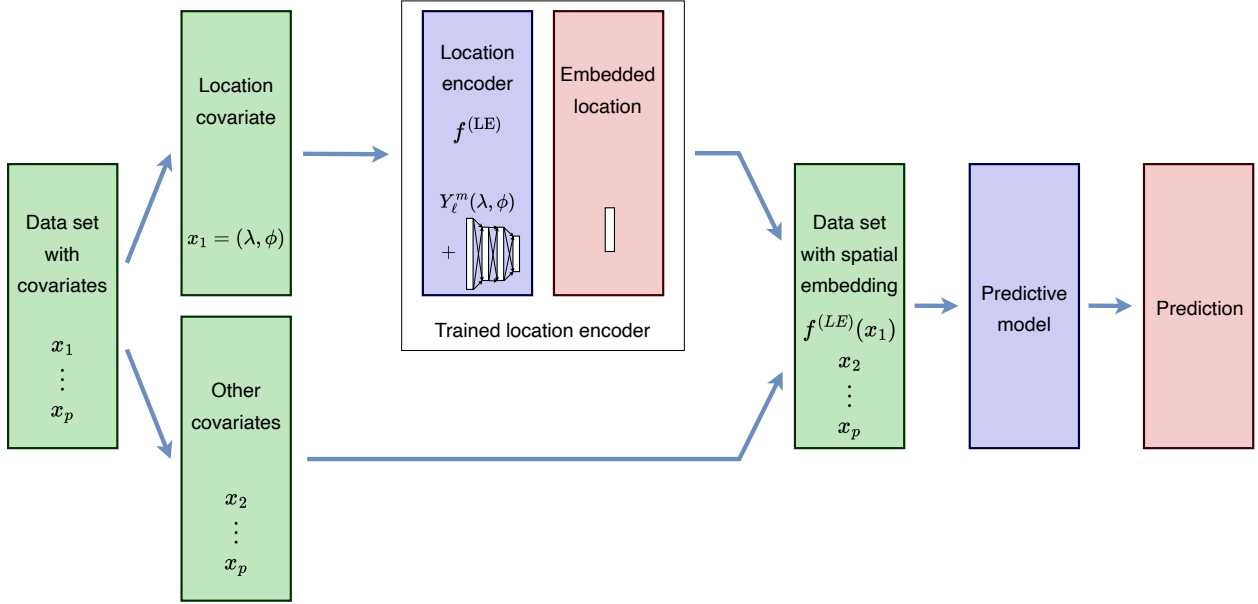


Figure 9: Application of the trained spatial embedding model in a case study where geographical coordinates are used as a covariate in a predictive model. By replacing the raw coordinate x_1 with its spatial embedding $f^{\text{LE}}(x_1)$, the model incorporates not only geographic position but also the spatial information observed at that location through the embedding.

values of d , d_{GS} , and d_{OSM} for each of the five embedding models, which are compared in the experimental study in Section 4.

The framework can be used in two ways. A first option is to apply the location encoder of one of the five trained models in Table 1, published on Hugging Face (Holvoet 2025), to any dataset with coordinates in Europe. A second option is to retrain the framework with spatial views tailored to a specific application, such as building characteristics from a commercial data provider or portfolio-specific exposure information.

All training is conducted using Python on a desktop computer with a single NVIDIA RTX 4070 GPU (12 GB), an Intel i9-13900K CPU, and 128GB of RAM. The implementation uses PyTorch and PyTorch Lightning for the construction and training of the embedding model, torchgeo and rasterio for the satellite imagery, and pandas and NumPy for the data handling. The GitHub repository at <https://github.com/freekholvoet/MultiviewSpatialEmbeddings> lists the complete set of dependencies with version numbers. Training the model on the full European dataset required approximately 12 hours, depending on hyperparameters. Calculating the embeddings for a new coordinate, as illustrated in Figure 9, takes under 1 millisecond.

4 Experimental study

We demonstrate the practical value of the spatial embeddings in two case studies. The first, in Section 4.1, predicts real estate prices in France. We use this application for two reasons. First, the dataset is publicly available and contains exact coordinates per observation, which supports the reproducibility of our results, whereas insurance claims data at this level of spatial granularity are

Model name	Embedding dimension d	GS spatial view	OSM spatial view
EU8_GS32_OSM32	8	$d_{\text{GS}} = 32$	$d_{\text{OSM}} = 32$
EU16_GS32_OSM16	16	$d_{\text{GS}} = 32$	$d_{\text{OSM}} = 16$
EU16_OSM16	16	$d_{\text{GS}} = 0$	$d_{\text{OSM}} = 16$
EU32_GS96_OSM32	32	$d_{\text{GS}} = 96$	$d_{\text{OSM}} = 32$
EU64_GS64	64	$d_{\text{GS}} = 64$	$d_{\text{OSM}} = 0$

Table 1: Overview of the five spatial embedding models used in the experimental study. Each model varies the overall embedding dimension d and the intermediate dimensions d_{GS} and d_{OSM} , while all other hyperparameters are fixed. Model acronyms are constructed as follows: the prefix **EU** indicates that the model is trained on the European dataset introduced in Section 2; this is followed by the overall embedding dimension d ; then **GS** with the dimension of the Google Satellite encoder d_{GS} ; and finally **OSM** with the dimension of the OpenStreetMap encoder d_{OSM} .

typically confidential. Second, the relationship between location and transaction prices is strong and well documented, which allows us to verify the spatial effects captured by the embeddings. The second case study, in Section 4.2, models flood claim counts across Belgian postal codes.

4.1 Case study on real estate pricing

4.1.1 Real estate transactions dataset

We use a real estate transaction dataset published by [Etablab \(n.d.\)](#), called the DVF data base. Our dataset contains 200 000 property sales in France between 2017 and 2023. Each record in the dataset describes one transaction with the sale price and descriptive variables about the property. Table 2 provides an overview of the variables in the dataset, including their descriptions and whether they are continuous or categorical.

The case study constructs a regression model using the variables in Table 2 as covariates with **price** as the response variable. The left-hand side of Figure 10 shows the distribution of the response variable in the dataset, and the right-hand side shows the average property value by postal code. All transaction prices lie between €100 000 and €2 000 000. The average transaction value of properties is €310 190, with 90% of properties having a transaction value between €112 000 and €800 000. The areas with a higher average transaction value include the Paris district, the south-east and south-west coast areas, and the French Alps bordering Switzerland. The centre of France features large areas with lower average transaction prices in the regions of Limousin, Auvergne, and Bourgogne, which are characterised by extensive areas of nature reserves and agricultural land. We observe a higher average transaction price in cities such as Orleans, Lyon, and Toulouse. The dataset does not contain properties in the departments Bas-Rhin, Haut-Rhin, and Moselle.

The transactions in the dataset date from 2017 to 2023, whereas the satellite imagery and OpenStreetMap tags used to train the embedding models were collected at the time of this study. The embeddings therefore describe the current spatial context of each location rather than the context

Variable	Original name (Etalab)	Description
price	valeur_fonciere	Transaction price at which the property was sold, as a continuous variable.
type	type_local	Type of property, as a categorical variable with three levels: apartment (type 1), industrial/commercial (type 2), or house (type 3).
land_use	code_nature_culture	Land-use classification code, as a categorical variable with 27 levels. See Appendix C for the codes and their meaning.
rooms	nombre_pieces_principale	Number of rooms in the property, as a continuous variable.
building_area	surface_reelle_bati	Surface area of the building in m^2 , as a continuous variable.
land_area	surface_terrain	Surface area of the land in m^2 , as a continuous variable.
date	date_mutation	Date of transaction, represented as three continuous variables: year, month, and day.
coordinates	latitude, longitude	Coordinates of the property, expressed as two continuous variables.

Table 2: Description of the variables in the French DVF real estate transaction dataset. The column *Variable* lists the variable names used in this paper, while the column *Original name (Etalab)* shows the dataset field names as defined in [Etalab \(n.d.\)](#). Each description specifies whether the variable is continuous or categorical. For categorical variables the levels are listed.

at the moment of sale. This temporal misalignment is a limitation of this specific demonstration and not of the framework. In practice, the embedding model can be retrained per calendar year, or per any other relevant period, with temporally matched data. Historical satellite imagery is available through Google Earth Engine ([Gorelick et al. 2017](#)), and past states of OpenStreetMap can be reconstructed from edit timestamps.

Figure 11 shows the distribution of the other variables in the French real estate dataset. The properties in the dataset are on average $135 m^2$, with 95% of properties being below $220 m^2$. The properties have on average four rooms, and a land area of $1012 m^2$. Around 81% of properties are classified as houses (Type 3), 12.5% are apartments (Type 1), and the others are industrial or commercial properties (Type 2). Most land use classifications are building lots (class S). More properties were sold in 2021 in comparison to 2020, and no sales were recorded in the latter half of 2022.

We denote the French real estate transaction dataset as $\mathcal{D}^{\text{FR}} = (\mathbf{x}_i, y_i)_{i=1}^{200\,000}$. Each index i represents one property in the dataset, with the vector $(x_{i,1}, \dots, x_{i,7})$ containing the value of each of the seven variables in the dataset, as described in Table 2, and y_i the price of that property. Let the value $x_{i,7}$ be the latitude-longitude coordinates of the location of property i . When using a spatial embedding model, we replace the latitude-longitude coordinates of each property with the spatial embedding vector representing those coordinates, constructed by the location encoder f^{LE} from a

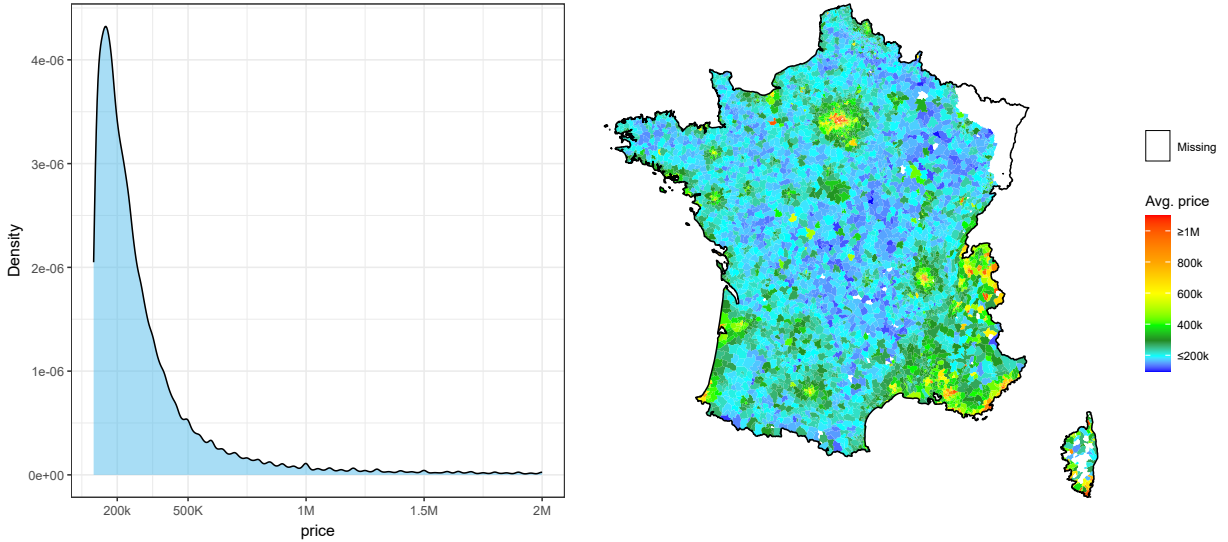


Figure 10: Plot of the property values in the dataset. The left-hand side shows a density plot of the property values. The right-hand side shows the average property value by postal code.

trained spatial embedding model defined in Section 3.4.

4.1.2 Predictive performance of spatial embeddings

We fit a regression model on the French real estate dataset with the transaction price as the response variable and the variables in Table 2 as input features. We will compare the effect on predictive accuracy when using the latitude and longitude coordinates as input features versus replacing latitude-longitude with its spatial embedding vector. We construct a generalised linear model (GLM), a generalised additive model (GAM), and a gradient boosting machine (GBM). The dataset is split into a randomly selected 80% of the observations as training data and the remaining 20% as an out-of-sample test set.

The embedding models in Table 1 are general-purpose models that are not fitted to the real estate dataset, and their construction is independent of the response variable. The contrastive pretraining of Section 3 aligns coordinates with spatial views and never observes a transaction price. The pretrained location encoder is a fixed transformation that is applied identically to the training and the test observations, comparable to any deterministic feature engineering step. This set-up is analogous to common practice in machine learning, where convolutional networks pretrained on ImageNet (Deng et al. 2009) or word embeddings such as word2vec (Mikolov et al. 2013) are reused as feature extractors for downstream tasks. The 80/20 split of the real estate data therefore provides a valid out-of-sample evaluation of the pricing models.

We fit the GLM on the log-transformed transaction price, using a Gaussian distribution with the identity link function. This specification corresponds to a lognormal regression model for the price itself, in line with the right-skewed price distribution in Figure 10. The categorical input features

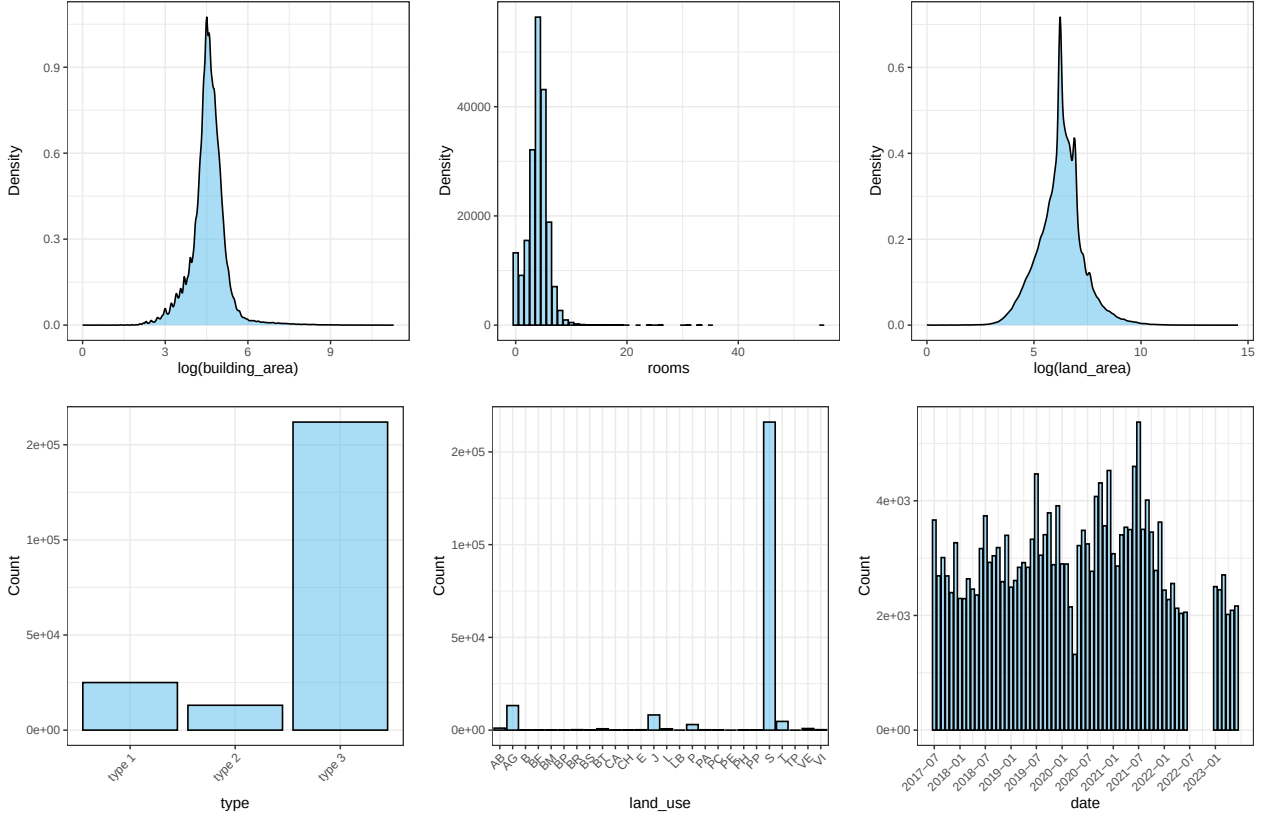


Figure 11: Exploratory data analysis of the French real-estate transaction dataset. From left to right, top row: density of $\log(\text{building_area})$, bar plot of `rooms`, and the density of $\log(\text{land_area})$. Bottom row: bar plot of `type`, bar plot of `land\_use`, and a bar plot showing the number of transactions per month over time.

are dummy-encoded for use in the GLM. The levels with the largest amount of exposure are taken as the reference level: `type1` and `land\_useS`. The location variable is used as two continuous variables, `latitude` and `longitude`, and the date variable is used as three continuous variables, `year`, `month`, and `day`. Variable selection is done via a bidirectional stepwise variable selection process; see [Hastie et al. \(2009\)](#). The full model structure of the GLM using latitude and longitude as input variables is as follows:

$$\begin{aligned} \mathbb{E}(\log(\text{price})) = & \beta_0 + \beta_1 \text{type}_2 + \beta_2 \text{type}_3 + \beta_3 \text{land_use}_{AB} + \dots + \beta_{28} \text{land_use}_{VI} \\ & + \beta_{29} \text{rooms} + \beta_{30} \text{building_area} + \beta_{31} \text{land_area} \\ & + \beta_{32} \text{year} + \beta_{33} \text{month} + \beta_{34} \text{day} + \beta_{35} \text{latitude} + \beta_{36} \text{longitude}. \end{aligned} \quad (3)$$

For the GLM using the spatial embedding as input variable, the terms $\beta_{35} \text{latitude} + \beta_{36} \text{longitude}$ are replaced with $\beta_{35} \text{spatial_embedding}_1 + \dots + \beta_{34+d} \text{spatial_embedding}_d$, where $(\text{spatial_embedding}_1, \dots, \text{spatial_embedding}_d)$ is the spatial embedding vector of dimension d .

We use a Gaussian GAM, fitted on the log-transformed transaction price with the identity link function as well. The categorical variables are used similarly to those in the GLM, with dummy encoding and the same reference levels. For the continuous variables `rooms`, `building_area`, and

`land_area`, univariate smooth functions f_1, f_2, f_3 are used. The variable `date` is treated as three continuous variables, `year`, `month`, and `day`, each with a univariate smooth function f_4, f_5, f_6 . The variable `coordinates` is used as two continuous variables, `latitude` and `longitude`, with a bivariate smooth function f_7 . All smooth functions are fitted with penalised cubic B-splines (Eilers and Marx 1996), fitted with the pyGAM library. The univariate smooth functions use 20 basis functions, and the bivariate smooth function on the coordinates is a tensor product of two marginal cubic B-spline bases with 10 basis functions each. The smoothing parameters are selected through a grid search minimising the generalised cross-validation criterion. The GAM with this bivariate smooth effect on the coordinates serves as a classical spatial smoothing benchmark against which the embedding based models are compared. The full model structure of the GAM is as follows:

$$\begin{aligned} \mathbb{E}(\log(\text{price})) = & \beta_0 + \beta_1 \text{type}_2 + \beta_2 \text{type}_3 + \beta_3 \text{land_use}_{\text{AB}} + \dots + \beta_{28} \text{land_use}_{\text{VI}} \\ & + f_1(\text{rooms}) + f_2(\text{building_area}) + f_3(\text{land_area}) \\ & + f_4(\text{year}) + f_5(\text{month}) + f_6(\text{day}) + f_7(\text{latitude}, \text{longitude}). \end{aligned} \quad (4)$$

For the GAM using the spatial embedding as input variable, the term $f_7(\text{latitude}, \text{longitude})$ is replaced by $f_7(\text{spatial_embedding}_1) + \dots + f_{6+d}(\text{spatial_embedding}_d)$.

For the GBM, we use the XGBoost implementation (Chen and Guestrin 2016). The categorical input features are one-hot encoded for use in the GBM. The GBM is fitted on the log-transformed transaction price with a squared error loss, in line with the assumptions made for the GLM and the GAM. The optimal tuning parameter values are determined via five-fold cross-validation. We tune the number of trees from the values $\{100, 500, 1000, 5000\}$, the depth of each tree from the values $\{3, 5, 7\}$, and the learning rate from the values $\{0.01, 0.001\}$. We assess model performance for the GLM, GAM, and GBM using the mean squared error (MSE) of log-transformed house prices on the test set. The same test set is used for all three models to ensure a fair comparison.

Table 3 shows the out-of-sample losses for the GLM, GAM, and GBM. The first column specifies how the location of the property was used in the model: either using latitude-longitude as input features (no spatial embedding), or a spatial embedding vector as an input feature, in which case the name of the embedding model follows the acronyms given in Table 1. We indicate the lowest out-of-sample loss for each column in bold. We observe that for the GLM, GAM, and GBM, using spatial embeddings instead of latitude-longitude coordinates as input features results in improved out-of-sample loss. The larger the dimension of the embeddings, the lower the out-of-sample loss, which is to be expected as larger embedding dimensions allow for more spatial information to be stored in the embeddings.

The embeddings improve all three models, but the improvement is more pronounced for the GLM and the GAM. Replacing the coordinates with the EU64_GS64 embedding reduces the out-of-sample loss by approximately 21% for the GLM and 25% for the GAM, and by approximately 7% for the GBM. These percentages are not directly comparable. The GBM with coordinate inputs already attains a lower out-of-sample loss than the GLM and the GAM achieve with spatial embeddings. It constructs a flexible spatial effect by repeatedly splitting on latitude and longitude, and extracts interactions between location and the other covariates from the large training set. Any further reduction is therefore harder to obtain. The embeddings still improve the out-of-sample loss of the GBM, resulting in the lowest value in Table 3.

The row OSM categories in Table 3 adds the counts of the six OpenStreetMap categories in the central H3 hexagon of each property as direct predictors, next to the latitude and longitude coor-

dinates. This baseline isolates the effect of giving the models direct access to the raw contextual spatial variables. The improvement over the coordinate only models is marginal, and every embedding model outperforms this baseline. The embeddings thus capture spatial structure beyond the local category counts by synthesising information from the surrounding hexagons and from the satellite imagery. The embedding workflow is also simpler in practice, since it only requires coordinates as input, without choosing an H3 resolution and extracting tag counts per observation.

The row AlphaEarth in Table 3 uses the pretrained 64-dimensional embeddings of the AlphaEarth foundation model (Brown et al. 2025), extracted from the annual satellite embedding collection in Google Earth Engine and matched to the transaction year of each property. They improve over the raw coordinates for the GLM and the GAM, but not for the GBM. Our EU64_GS64 model has the same dimension and has a lower out-of-sample deviance for all three model classes. AlphaEarth encodes the earth surface for general purpose use at global scale, whereas our framework encodes spatial views chosen for the application at hand, focussed on the European continent.

Spatial treatment	GLM	GAM	GBM
No spatial embedding	0.3122	0.2362	0.1415
OSM categories	0.3099	0.2330	0.1382
AlphaEarth	0.2576	0.2306	0.1754
EU8_GS32_OSM32	0.3070	0.2147	0.1344
EU16_GS32_OSM16	0.2861	0.1978	0.1332
EU16_OSM16	0.2775	0.1929	0.1328
EU32_GS96_OSM32	0.2646	0.1829	0.1325
EU64_GS64	0.2473	0.1762	0.1310

Table 3: Comparison between models using latitude–longitude and spatial embedding as input features. Results show out-of-sample losses, measured in mean-squared error on the log transform of the transaction price. For spatial treatment, the model acronyms are used as defined in Table 1. The lowest out-of-sample loss per column is indicated in bold.

4.1.3 Model interpretation of spatial effect

Variable importance We use the permutational variable importance tool from Olden et al. (2004) to compare the importance of each variable in our dataset on the prediction of real estate price. The variable importance is measured as the average change in prediction when randomly permuting the values of an input feature. For a prediction model $f_{\text{pred}}(\cdot)$, and an input feature x_j in the dataset $\mathcal{D}^{\text{FR}}$, the importance of that feature is defined as

$$\text{VIP}_{x_j} = \frac{1}{|\mathcal{D}^{\text{FR}}|} \sum_{\mathbf{x}_i \in \mathcal{D}^{\text{FR}}} \text{abs}(f_{\text{pred}}(x_{i,1}, \dots, x_{i,j}, \dots, x_{i,7}) - f_{\text{pred}}(x_{i,1}, \dots, \tilde{x}_{i,j}, \dots, x_{i,7})), \quad (5)$$

where $\tilde{x}_{i,j}$ is a random permutation of all observed values of that input variable. When calculating the variable importance of the property’s location, the latitude–longitude pairs are permuted together rather than the latitude and longitude values separately. Similarly, when using the spatial

embedding to represent location, the entire embedding vectors are permuted, not the individual elements within each vector. We calculate the variable importance for each of the input variables in Table 2. Figure 12 shows the variable importance for the GLM on the left, the GAM in the middle, and the GBM on the right. Dark blue shows the effect of the model with spatial embeddings, whereas light blue shows the effect of the model using latitude-longitude coordinates. For the spatial embedding, we utilised the location encoder from the trained EU32_GS96_OSM32 model, as described in Section 3.4. We observe in the GLM that the coordinates of the property are the third most important variable. This is because the GLM can only fit two coefficients, one for latitude and one for longitude, which is not sufficient to capture any nuanced spatial effects. Using spatial embeddings, however, the embedding vector is seen as the most important variable in the model. For both the GAM and the GBM, location is the most important variable, whether used via latitude and longitude coordinates or spatial embeddings. For the GLM and the GAM, the effect of permuting the spatial embeddings is so significant that the relative importance of other variables is no longer visible in Figure 12. This is not the case for the GBM, where the importance of non-location variables stays relatively equal between the model using latitude and longitude versus the model using spatial embeddings.

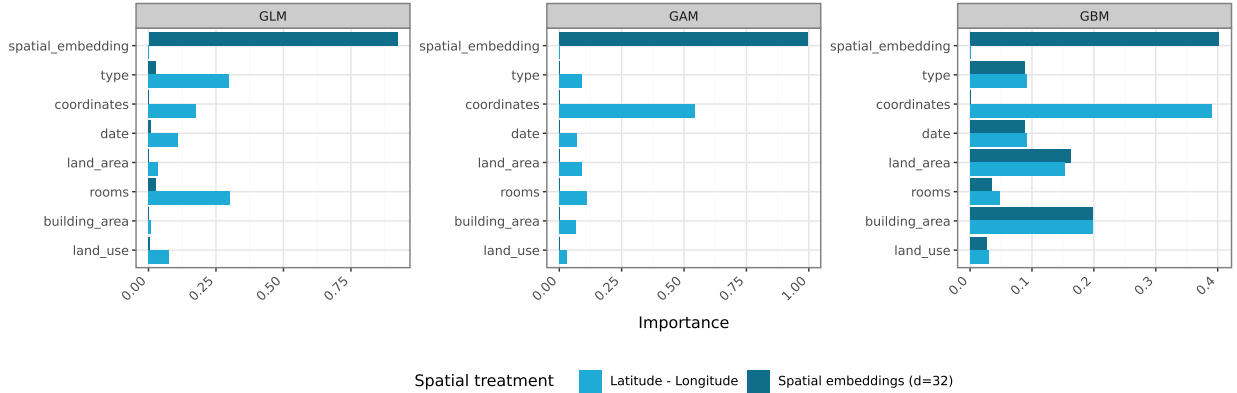


Figure 12: Variable importance as measured in the GLM (left side), the GAM (middle), and the GBM (right side). In dark blue, we show the importance of each variable as measured in the model with spatial embeddings. In light blue, we show the importance measured in the model using latitude-longitude coordinates.

Partial dependence We use partial dependence plots (Hastie et al. 2009) to examine the relationship between the predicted real estate price and the property’s location in the pricing model. For a prediction model $f_{\text{pred}}(\cdot)$, where input feature $x_{i,7}$ represents the latitude-longitude coordinates, we calculate the partial dependence effect of a location (λ, ϕ) as

$$\text{PD}(\lambda, \phi) = \frac{1}{|\mathcal{D}^{\text{FR}}|} \sum_{\mathbf{x}_i \in \mathcal{D}^{\text{FR}}} f_{\text{pred}}(x_{i,1}, \dots, x_{i,6}, (\lambda, \phi)). \quad (6)$$

When using a prediction model with the spatial embedding of latitude-longitude coordinates as input feature, as calculated via the encoder f^{LE} , the partial dependence effect of the location (λ, ϕ) is calculated as

$$\text{PD}(\lambda, \phi) = \frac{1}{|\mathcal{D}^{\text{FR}}|} \sum_{\mathbf{x}_i \in \mathcal{D}^{\text{FR}}} f_{\text{pred}}(x_{i,1}, \dots, x_{i,6}, f^{\text{LE}}(\lambda, \phi)). \quad (7)$$

Let $\{(\lambda_c, \phi_c), c = 1, \dots, 6048\}$ denote the latitude–longitude coordinates of the centroids of the 6 048 postal codes in France. For each coordinate pair, we compute the partial dependence effect, which results in a spatial map showing how location influences the predicted real estate price.

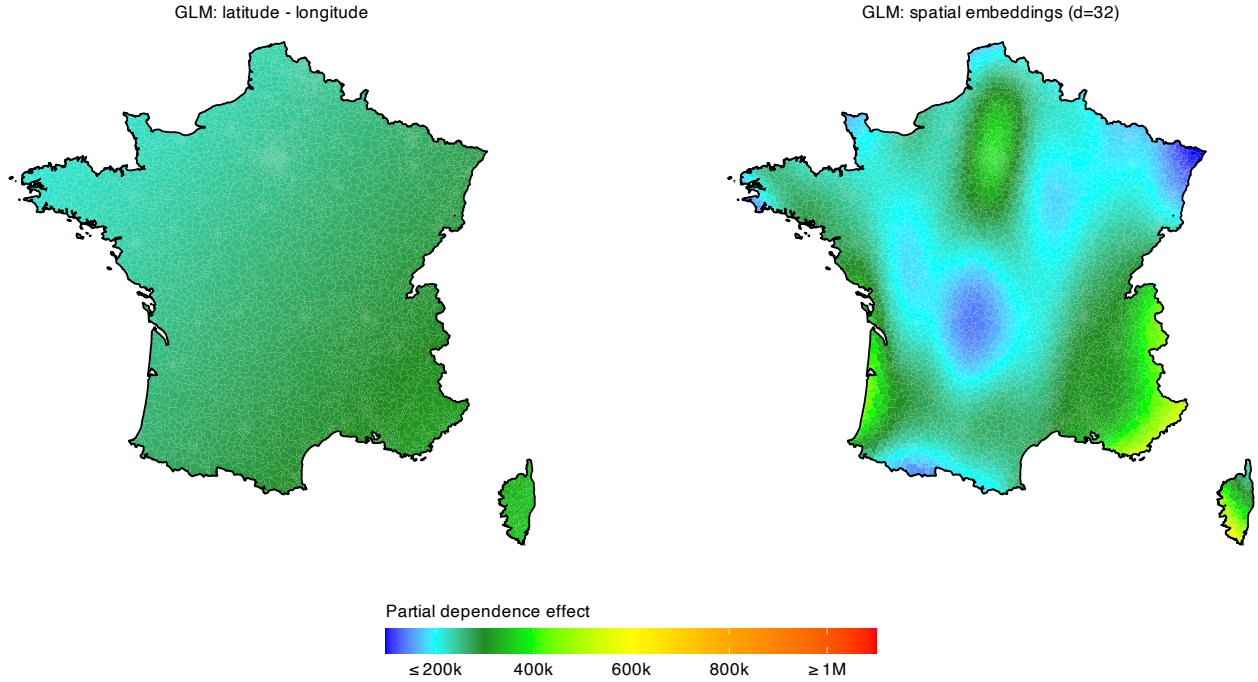


Figure 13: Partial dependence effect of location on real estate price, calculated per postal code for predictions from the GLMs. The left-hand side is the model using latitude–longitude coordinates as input features, and the right-hand side is the model using spatial embeddings instead of latitude–longitude. We use the EU32_GS96_OSM32 model to extract spatial embeddings.

Figure 13 shows the partial dependence effect as calculated with the GLM. The left-hand side shows the effect of the GLM using latitude and longitude as input features as defined in Equation (3); the right-hand side shows the GLM using the spatial embedding vector as input features. The GLM using latitude and longitude shows a negligible partial dependence effect, with each postal code having a very similar average predicted price. This is in line with the variable importance plots in Figure 12, which indicate that the variables latitude and longitude are not important in this GLM. By contrast, the partial dependence effect for the GLM using spatial embeddings shows higher predicted prices around Paris and in the south-east and south-west regions of France, with a further effect visible on the island of Corsica. This demonstrates that the GLM can capture more nuanced patterns through spatial embeddings than it can solely based on latitude and longitude coordinates.

Figure 14 shows the partial dependence effect per postal code as calculated with the GAM. The left-hand side displays the GAM using latitude and longitude as input, as described in Equation (4). Compared to the GLM, the bivariate smoothing on the coordinates produces a more nuanced

spatial pattern, with higher predicted prices around Paris and in the south-east and south-west regions. The right-hand side shows the GAM using spatial embeddings as input. In this case, the model captures more complex spatial variation, distinguishing between urban centres and rural areas.

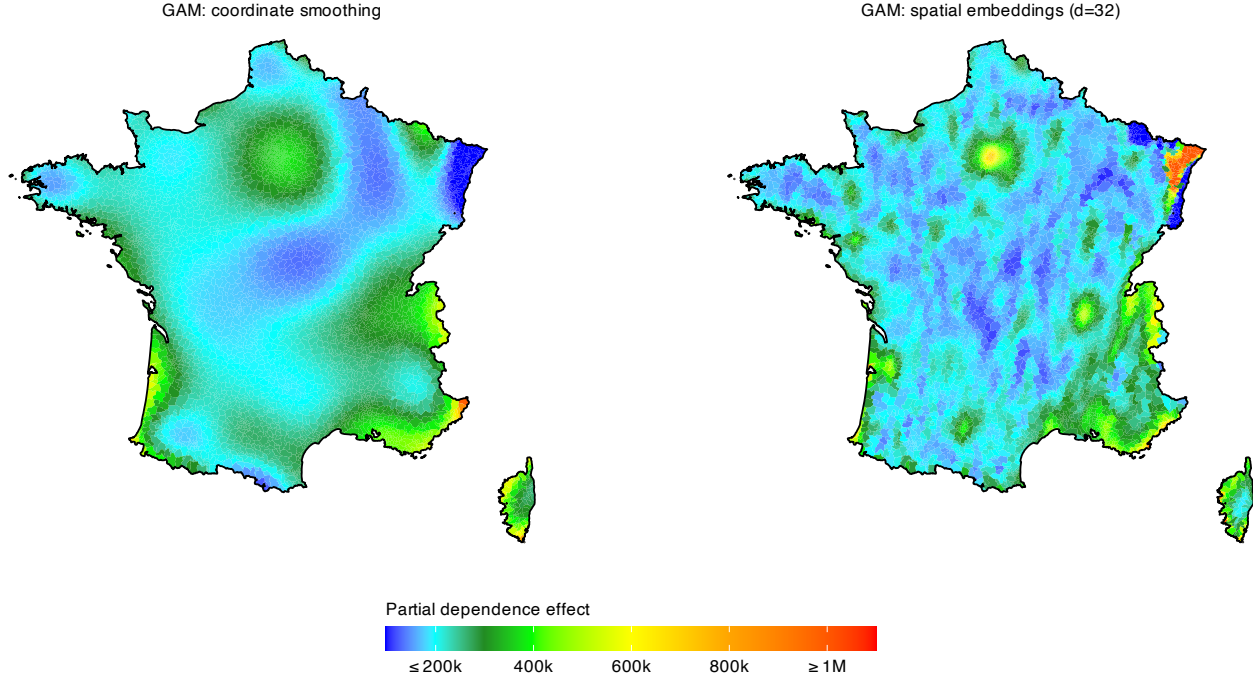


Figure 14: Partial dependence effect of location on real estate price, calculated per postal code for predictions from the GAM. The left-hand side is the model using latitude-longitude coordinates as input features, and the right-hand side is the model using spatial embeddings instead of latitude-longitude. We use the EU32_GS96_OSM32 model to extract spatial embeddings.

Figure 15 shows the partial dependence effect of the GBM using latitude-longitude as input features on the left and using the spatial embeddings on the right-hand side. For both the GBM using latitude-longitude and the GBM using spatial embeddings, we see a difference between cities in France and rural areas. The GBM using latitude-longitude, however, shows a square pattern caused by the linear splits made by the regression trees in the GBM (Henckaerts et al. 2021; Geerts et al. 2024). The partial dependence effect from the GBM using spatial embeddings does not show these square artifacts and shows the shapes of cities more clearly. At the scale of Figure 15, both GBMs thus recover the dominant spatial pattern, with higher predicted prices in and around the cities and lower predicted prices in the rural centre. This similarity breaks down at finer spatial scales. The splits on latitude and longitude divide the map into rectangular regions with a constant predicted effect, so the coordinate-based GBM cannot represent spatial variation within such a region.

Fine-grained partial dependence effects Instead of one coordinate pair per postal code, we now define a fine-grained set of locations in the area between Lyon and Switzerland. We randomly sample a set of 10 000 locations in this area. By calculating the partial dependence effect for this set of locations, we identify fine-grained effects in the relationship between location and predicted prices. We calculate the partial dependence effect for each location following Equation (7) using

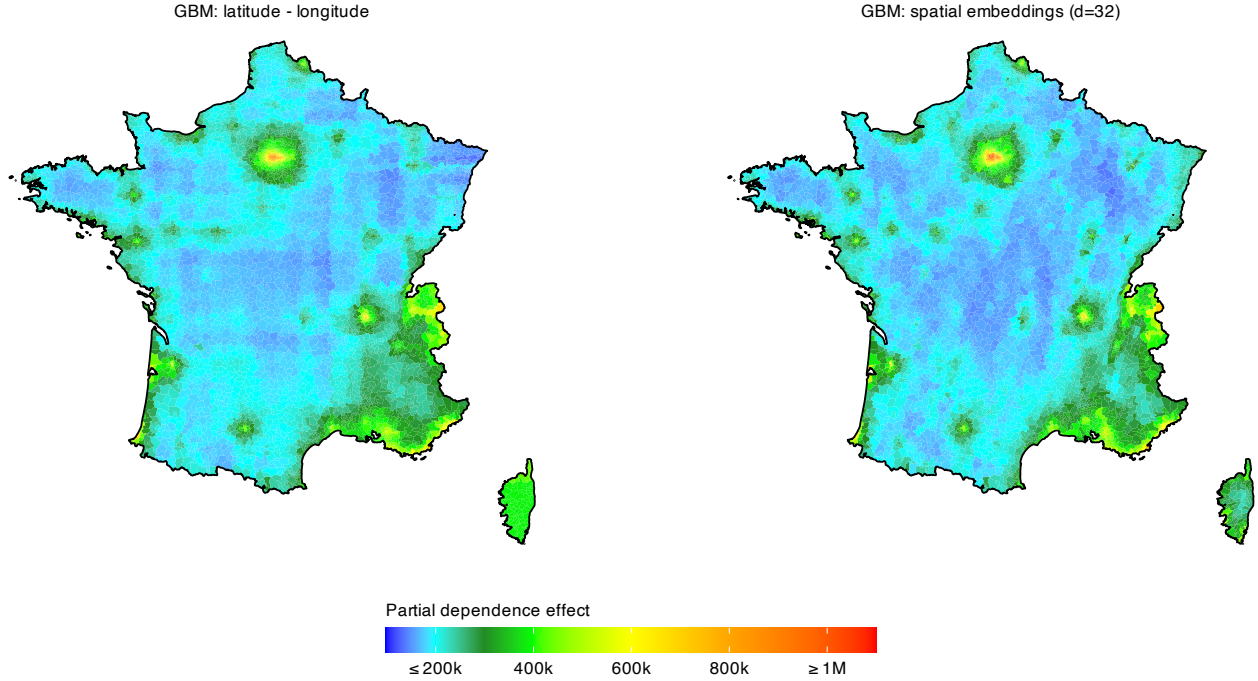


Figure 15: Effect of location on average predicted real estate price, calculated per postal code for predictions from the GBMs. The left-hand side is the model using latitude-longitude coordinates as input features, and the right-hand side is the model using spatial embeddings instead of latitude-longitude. We use the EU32_GS96_OSM32 model to extract spatial embeddings.

the GBM.

Figure 16 shows the partial dependence effects. The left-hand side displays results at the postal code level, while the right-hand side shows the fine-grained set of locations. Three localised patterns appear in the fine-grained map that are not visible at the postal code level, highlighted by red ovals. The first is a small, oblong area with higher predicted prices, corresponding to the touristically popular northwestern slopes of the Aravis range. The second is a wide oblong area with lower predicted prices, coinciding with the La Lauzière range. The La Lauzière mountain range is less touristic and houses only a few resorts or ski areas. The third is an angular pattern of higher predicted prices matching the Les Trois Vallées area. This is the largest connected ski area in the world and is highly touristic. These examples demonstrate that the GBM has fitted a relationship between real estate prices and the spatial information captured in the spatial embeddings.

Generalisation of spatial effects to regions without training observations We examine how the spatial effect fitted by the GBM generalises to regions without price observations in the training data. The departments Bas-Rhin, Haut-Rhin, and Moselle are absent from the French real estate dataset, since transactions in these departments are not reported to the DVF database. We compare the partial dependence of the coordinate-based and the embedding-based GBM for postal codes located in these departments. The embedding model has observed the region during its pretraining, since the spatial views are available there, but the pricing model has never observed any property there.

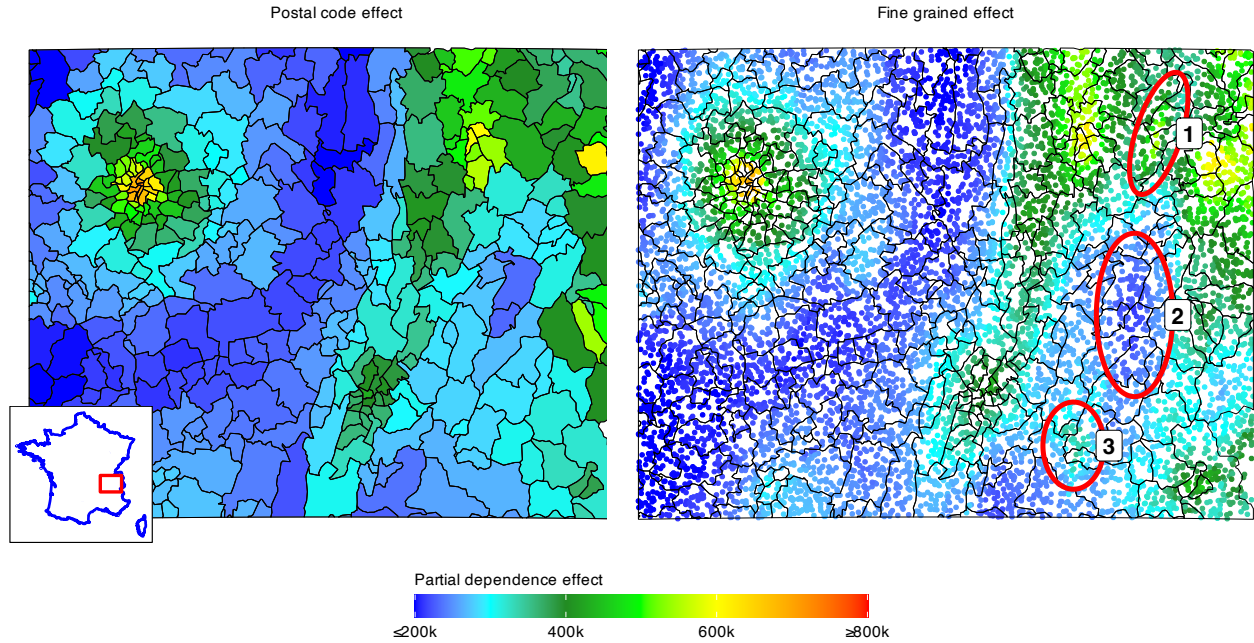


Figure 16: Partial dependence effect from the GBM using spatial embeddings calculated in a selected area of France between Lyon and Switzerland. The used spatial embedding model is the EU32_GS96_OSM32 model. The selected area is indicated on the small map of France on the bottom left. The left-hand side shows the partial dependence effect calculated on the level of postal codes, and the right-hand side shows the partial dependence effect calculated on 10 000 randomly sampled points in the selected area.

The first panel of Figure 17 shows the empirical mean transaction price per postal code in the region between Nancy and the German/Swiss border; a map of France in the lower left highlights the selected area. A thick grey line marks the spatial extent of the training data, and postal codes in white indicate areas absent from the dataset. The second panel shows the partial dependence (PD) for this region obtained from the GBM using latitude and longitude as inputs. Because the excluded departments lie beyond the range of latitude and longitude values present in the training data, the GBM cannot form new splits in these areas. Instead, the model extrapolates by assigning these unseen locations to the nearest regions learnt during training, resulting in an unrealistic, piecewise-constant partial dependence effect. The third panel shows the partial dependence effect obtained from the GBM using spatial embeddings as input features. Here, the model captures a lower partial dependence effect in the central part of the region and a higher effect along the German border. Both patterns are intuitive: the central area is predominantly rural with lower population density, whereas larger cities, such as Strasbourg, are located along the border. By using spatial embeddings, the model associates predictions with spatial characteristics rather than with absolute coordinates. This representation allows the fitted spatial effect to generalise to regions without training observations by exploiting similarities in spatial structure. The partial dependence pattern, rather than the absolute price level, is validated here, since no transaction prices are available in the excluded departments.

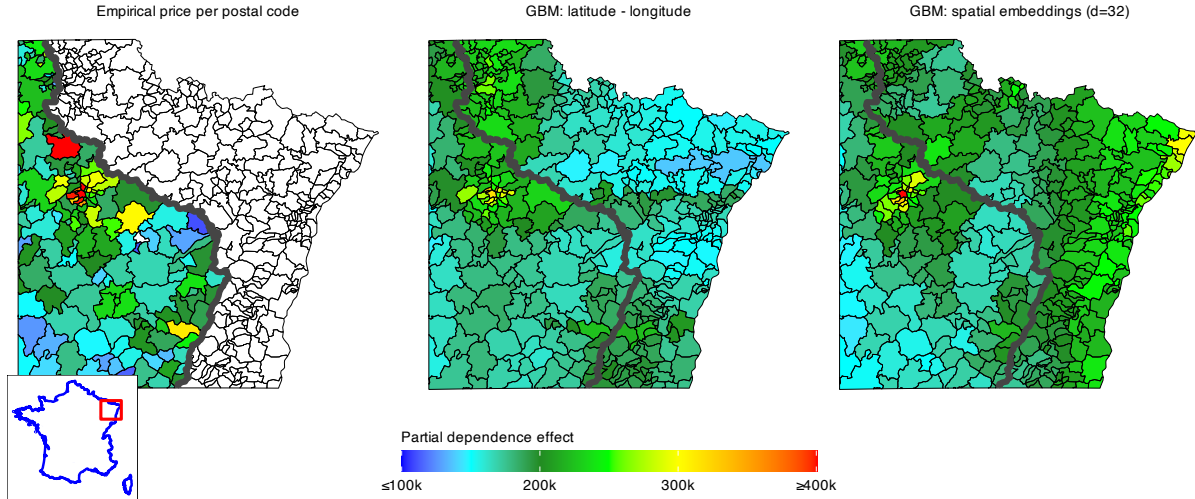


Figure 17: Selected area in France between the city of Nancy and the German border, as indicated on the inset map (bottom left). The left panel shows the empirically observed average transaction price per postal code, where white corresponds to postal codes without observations in the dataset, namely in the departments Bas-Rhin, Haut-Rhin, and Moselle. The middle panel shows the partial dependence effect calculated with the GBM using latitude and longitude as input. The right panel shows the partial dependence effect from the GBM using spatial embeddings. The grey line marks the boundary between regions included in the training data and regions without observations.

4.2 Insurance case study: flood claim counts in Belgium

We perform a second case study on insurance claim counts, using the flood claims recorded in the Klimaatschademonitor ([Assuralia 2025](#)) of Assuralia, the federation of Belgian insurance companies. The dataset contains the number of claims per Belgian postal code and per calendar year, for the period 2015 to 2025 and separately for the perils vehicle damage, flood, and storm. We focus on the flood peril. Insurance datasets with claims linked to policies at the coordinate or address level are proprietary, but this open dataset allows us to construct a reproducible insurance case study. The dataset contains 1 146 postal codes and 12 606 postal code and calendar year combinations. The members of Assuralia represent nearly the entire Belgian insurance market, and flood damage coverage is a mandatory part of Belgian fire insurance policies. Hence, the recorded claims effectively span the insured building stock of the country. There is no exposure measure in the dataset, such as the number of policies or insured buildings per postal code. The models therefore estimate the expected number of flood claims per postal code and calendar year, rather than a claim frequency per unit of exposure, and the spatial variation they capture reflects both the concentration of insured buildings and the underlying flood risk.

Figure 18a shows the total number of flood claims per calendar year and Figure 18b the observed average annual number of claims per postal code. The annual claim counts per postal code are heavily right-skewed. About 35% of the observations record no flood claims, the average count per

year and per postal code is 11.9 claims. The maximum observed number is 3 401 claims, recorded in postal code 4800 (Verviers) during the July 2021 floods in the east of Belgium. The year 2021 alone accounts for 55% of all flood claims in the dataset, more than three times the total of any other year, as seen in Figure 18a. The spatial pattern in Figure 18b concentrates along the Meuse and Vesdre valleys in the east of the country.

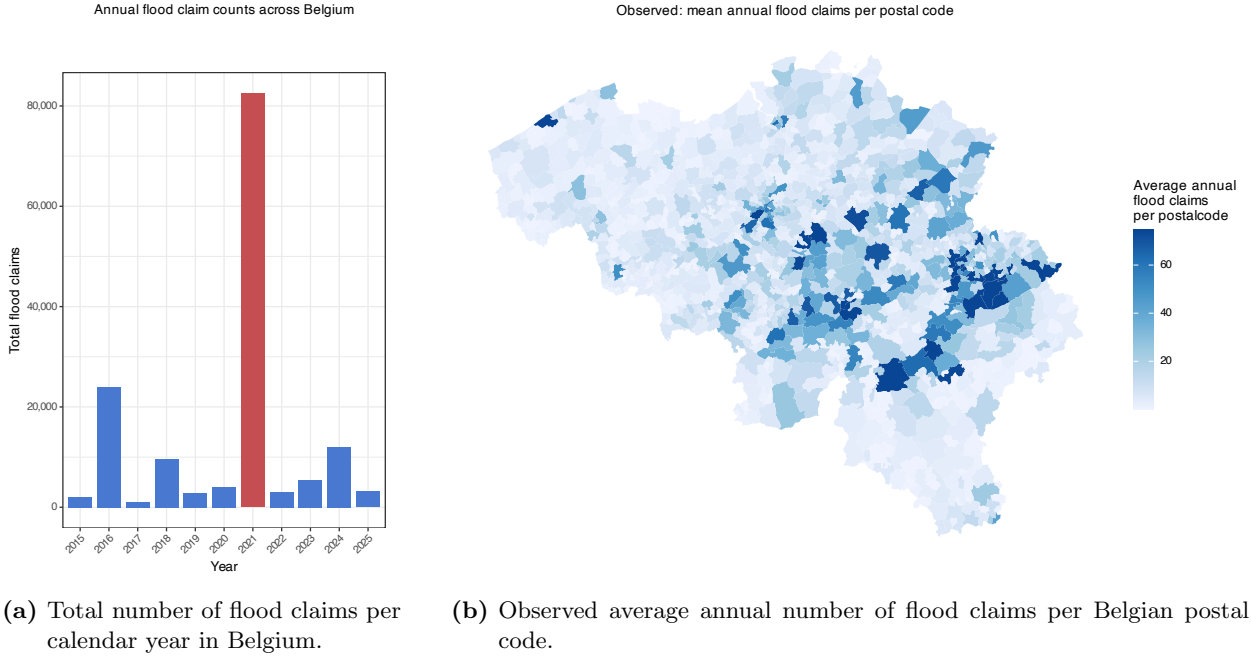


Figure 18: Flood claims recorded in the Klimaatschademonitor of Assuralia ([Assuralia 2025](#)), for the period 2015 to 2025. Panel (a) shows the annual totals across Belgium. Panel (b) shows the average annual number of claims per postal code.

We denote the Belgian flood dataset as $\mathcal{D}^{\text{BE}} = (\mathbf{x}_i, y_i)_{i=1}^{12\,606}$, where each index i represents one postal code and calendar year combination, y_i the observed number of flood claims, and $\mathbf{x}_i$ the location of the postal code centroid. The location is used either as the latitude and longitude coordinates or as the 32-dimensional spatial embedding obtained by evaluating the location encoder of the trained EU32_GS96_OSM32 model at the centroid. The postal code itself is not used as a categorical variable. We compare a Poisson GLM, a Poisson GAM, and a Poisson GBM, all with a logarithmic link function. The GLM uses the latitude and longitude coordinates, or the 32 embedding components, as linear terms. The GAM uses a bivariate smooth function of latitude and longitude, or a univariate smooth function per embedding component, fitted with penalised cubic B-splines. The GBM uses the XGBoost implementation ([Chen and Guestrin 2016](#)) with the Poisson deviance as loss function, 500 trees of depth 4, a learning rate of 0.05, and a subsample fraction of 0.8. The calendar year is not used as a predictor, so each model estimates an average annual number of flood claims per postal code.

Annual flood claim counts vary strongly between calendar years, driven by individual events such as the July 2021 floods in Belgium. A temporal train and test split would therefore mainly measure calendar year volatility rather than the quality of the territorial risk classification. We instead use five-fold cross validation grouped by postal code. We assign the 1 146 postal codes at random to

five sets of approximately equal size. Each set serves once as the hold-out set, while the models are fitted on the postal codes in the other four set. Grouping by postal code means that all calendar year observations of a postal code belong to the same set. Each postal code then gets a prediction by a model that was not fitted on any of its observations, which directly tests the assignment of risk levels to new territories. We calculate the out-of-sample Poisson deviance, averaged over the five sets, for each model. For a model f and a set of n hold-out observations, the Poisson deviance is calculated as

$$D_{\text{Poisson}}(f(\mathbf{x}), y) = \frac{2}{n} \sum_{i=1}^n \left(y_i \ln \frac{y_i}{f(\mathbf{x}_i)} - (y_i - f(\mathbf{x}_i)) \right), \quad (8)$$

with the convention $0 \ln 0$ is put equal to zero for observations without claims.

Table 4 shows the out-of-sample Poisson deviance for the three model classes. The model using spatial embeddings attains a lower deviance than those using coordinates for all three model classes. The GLM with spatial embeddings attains a deviance below that of both the GAM with a bivariate coordinate smoother and the GBM on raw coordinates. The deviance is dominated by a small number of postal codes with extreme claim counts, given the heavy right skew. It measures how close the predicted claim counts are to the observed counts, rather than how well a model separates the high-risk postal codes from the low-risk ones.

Feature set	GLM	GAM	GBM
Coordinates	52.31	49.60	49.99
Spatial embedding	49.42	49.35	48.88

Table 4: Out-of-sample Poisson deviance for the flood claim count models, averaged over the five sets of the cross-validation grouped by postal code. The lowest value is indicated in bold. The spatial embedding is extracted with the EU32_GS96_OSM32 model.

Figure 19 shows the predicted average annual number of flood claims per postal code. The GLM on the coordinates produces a smooth gradient from west to east, since the two coefficients can only tilt the predicted counts in a single direction. The GAM on the coordinates adds one broad area of higher predicted counts in the east, without separating the postal codes within that area. The GBM on the coordinates distinguishes areas of higher and lower predicted counts, with strong contrasts between neighbouring postal codes. The models using the spatial embeddings separate areas of high and low predicted counts for all three model classes. They locate the highest predicted counts around Verviers and Liège, in line with the observed pattern in Figure 18b. The GLM on the spatial embeddings thus recovers the observed flood pattern more closely than the GAM with a bivariate smoother on the coordinates, which spreads the risk over a single broad region in the east. The GLM and the GAM on the spatial embeddings produce nearly the same map, and both vary more smoothly over neighbouring postal codes than the GBM.

The model predictions do not depend on the calendar year, so each postal code receives a single predicted annual claim count. We denote by $\hat{y}_p$ the predicted average annual number of flood claims for postal code p under a given model, and by $\bar{y}_p$ the corresponding observed average over the calendar years. Following Holvoet et al. (2025), we compare the risk classification achieved by

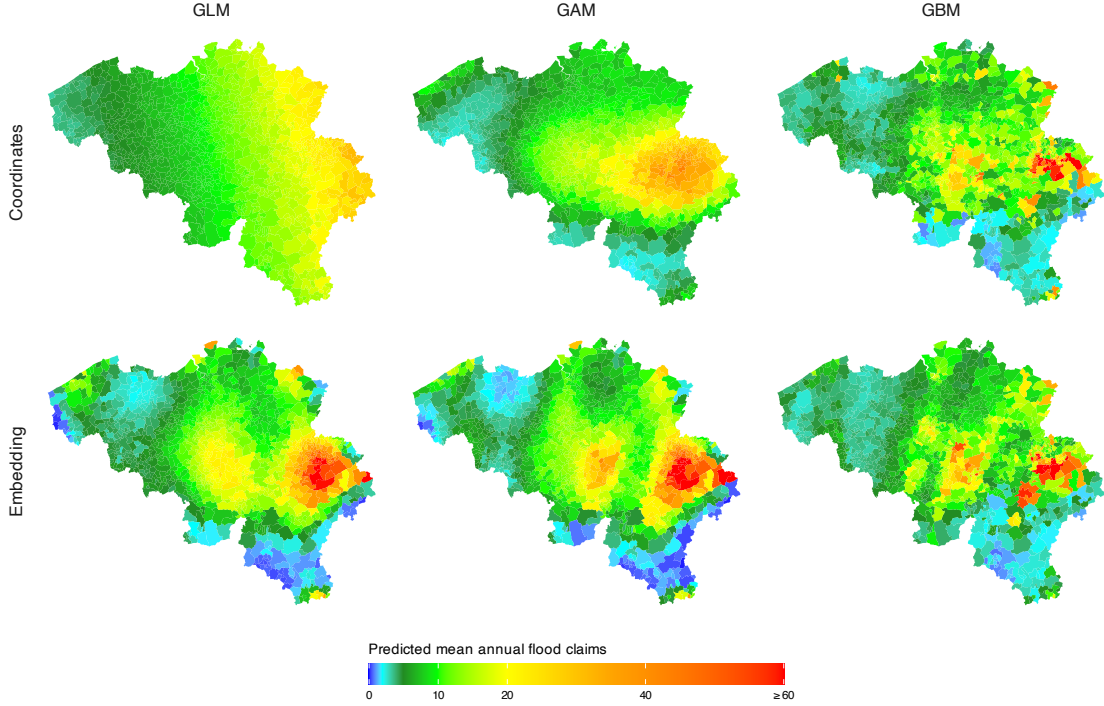


Figure 19: Predicted average annual number of flood claims per Belgian postal code, for the GLM (left), the GAM (middle), and the GBM (right). The top row shows the models using the latitude and longitude coordinates of the postal code centroid, the bottom row the models using the spatial embeddings of that centroid. All predictions are made out of sample. The spatial embeddings are extracted with the EU32_GS96_OSM32 model.

the models using Lorenz curves (Lorenz 1905). For a model f , let F_n be the empirical cumulative distribution function of the predictions $\hat{y}_p$ over the $n = 1\,146$ postal codes, and $r_p^f = F_n(\hat{y}_p)$ the risk score of postal code p . The Lorenz curve evaluated in $s \in [0, 1]$ is

$$\text{LC}^f(s) = \frac{\sum_p \bar{y}_p \mathbb{1}\{r_p^f \leq s\}}{\sum_p \bar{y}_p}. \quad (9)$$

The Lorenz curve shows the accumulation of observed claims, ordered by the risk score obtained with model f . A curve further away from the line of equality represents a better risk classification. Figure 20 shows the Lorenz curves of the out-of-fold predictions. Within each model class, the curve of the model using the spatial embeddings lies further away from the line of equality than the curve of the model using the coordinates. The GBM using the spatial embeddings lies furthest away, consistent with its lowest out-of-fold deviance in Table 4.

We use the ordered Lorenz curve, introduced by Frees et al. (2011), to directly compare the risk classification under one model relative to another. For two models f_A and f_B with predictions $\hat{y}_p^A$ and $\hat{y}_p^B$ for postal code p , we define the relativities of model B over model A as

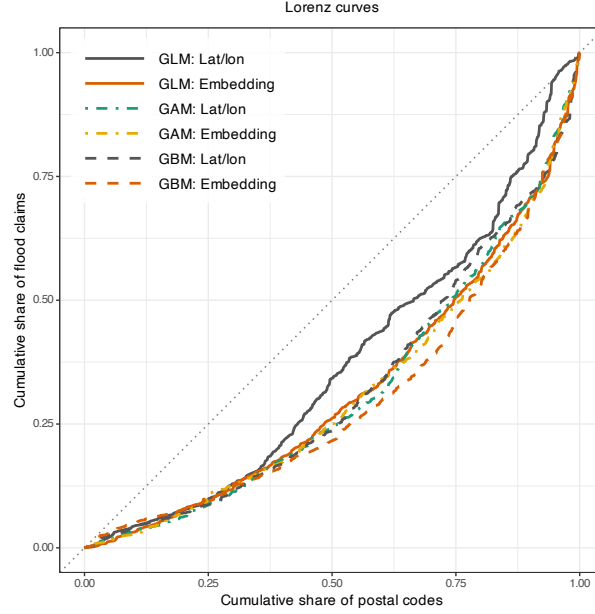


Figure 20: Lorenz curves for the flood claim count models, based on the out-of-fold predictions per postal code. A grey dotted line shows the line of equality.

$$\text{rel}_p^{(A,B)} = \frac{\hat{y}_p^B}{\hat{y}_p^A}. \quad (10)$$

With F_n the empirical distribution function of the relativities $\text{rel}_p^{(A,B)}$ over all postal codes, the ordered Lorenz curve is defined as

$$\left(\frac{\sum_p \hat{y}_p^A \mathbb{1} \left\{ F_n \left(\text{rel}_p^{(A,B)} \right) \leq s \right\}}{\sum_p \hat{y}_p^A}, \frac{\sum_p \bar{y}_p \mathbb{1} \left\{ F_n \left(\text{rel}_p^{(A,B)} \right) \leq s \right\}}{\sum_p \bar{y}_p} \right), \quad (11)$$

over $s \in [0, 1]$. The ordered Lorenz curve illustrates the potential improvement in risk classification an insurer can achieve by switching from model f_A to model f_B . The Gini index is equal to twice the area between the line of equality and the curve, and a Gini index close to zero indicates that model f_B does not improve on model f_A (Frees et al. 2011). Table 5 presents the Gini indices for all pairs of models. Following the min-max strategy of Frees et al. (2014), we select the model A with the lowest maximum Gini index, since this model leaves the least potential for improvement in risk classification. This criterion selects the GBM with spatial embeddings, with a maximal Gini index of 15.86.

The July 2021 floods dominate the claim totals in Figure 18a and could in principle drive the rankings on their own. We therefore repeat the full evaluation, including the refitting of all models, on the data without calendar year 2021, using the same set assignment. Within each model class, the model using the spatial embeddings retains the lower out-of-sample deviance, with 18.68 against 19.27 for the GLM, 18.68 against 18.73 for the GAM, and 18.34 against 18.82 for the GBM. The

		Model B					
		GLM coordinates	GLM embedding	GAM coordinates	GAM embedding	GBM coordinates	GBM embedding
Model A	GLM coordinates		29.54	26.84	31.06	29.39	33.38
	GLM embedding	4.61		4.23	10.32	15.84	20.47
	GAM coordinates	0.09	14.97		17.54	18.46	22.80
	GAM embedding	5.28	3.35	6.55		13.73	17.29
	GBM coordinates	13.18	16.38	16.28	17.25		18.86
	GBM embedding	9.80	15.86	14.52	14.82	9.96	

Table 5: Gini indices, in percent, calculated as twice the area between the ordered Lorenz curve and the diagonal for model B relative to model A, computed on the out-of-fold predictions per postal code. For each row, the maximum Gini index is given in bold. The min-max strategy selects the model A with the lowest maximum Gini index, shown by the highlighted cell and underlined model.

maximum Gini index per model is lower without this year, which shows that the extreme year carries a substantial part of the spatial signal. The min-max strategy again selects the GBM with spatial embeddings, with a maximal Gini index of 8.77. The ranking improvements are thus not an artefact of a single extreme event.

This case study demonstrates the added value of spatial embeddings in an actuarial setting, using open data at the postal code resolution. The embeddings improve the out-of-sample territorial risk classification of the GLM, the GAM, and the GBM on flood claim count modelling. The postal code centroids used in this case study only capture part of the spatial resolution of the embeddings, since a single point represents an entire postal code area. We expect more substantial gains with granular policy-level data within an insurance company, where exact building locations are available and the number of policies per location enters the Poisson models as an exposure offset.

5 Conclusion

In this paper, we introduced a framework for constructing spatial embeddings using contrastive learning in a multi-view setting. To demonstrate the multi-view spatial embedding model, we combined satellite imagery and OpenStreetMap tags to learn low-dimensional representations of geographic locations. These embeddings are trained to align with coordinate-based encodings, allowing any dataset with latitude–longitude information to be enriched with spatial context without requiring direct access to the original spatial inputs. The paper is accompanied by a GitHub repository at <https://github.com/freekholvoet/MultiviewSpatialEmbeddings>, which provides the code for training the spatial embedding model. The trained models defined in Table 1 are available in Holvoet (2025).

We demonstrated the practical value of these embeddings in a real estate pricing application and

in an insurance case study on flood claim counts in Belgium. Replacing raw coordinates with spatial embeddings consistently improved predictive performance across GLMs, GAMs, and GBMs. The embeddings captured spatial structure more effectively than traditional location features and provided post-hoc explainable spatial effects. Variable importance and partial dependence analyses showed that the embeddings learnt meaningful geographic patterns, including urban–rural differences, regional economic variation, and fine-grained effects related to topography and land use. These effects are difficult to capture using raw coordinates or categorical location indicators such as postal codes. Moreover, the use of spatial embeddings enabled models to generalise their fitted spatial effects to regions without outcome observations in the training data, by associating predictions with spatial characteristics rather than coordinates. This property is particularly relevant for insurance applications, where actuaries often need to assess risk in new or underrepresented areas. The real estate case study was chosen deliberately as the primary demonstration. The relationship between location and transaction prices is strong and well understood, which makes the partial dependence effects directly verifiable against known geography, with high prices around Paris and in popular ski areas and lower prices in the rural centre of France. Such verification would be far more difficult on a noisier insurance outcome. The flood case study in Section 4.2 complements this verification with direct evidence on insurance data, where the embeddings improve the out-of-sample risk classification of flood claims across Belgian postal codes. Together with existing applications of learnt spatial representations in actuarial science, such as claim frequency modelling for motor and home insurance (Blier-Wong et al. 2022), these results support the use of spatial embeddings across a broad range of insurance applications. A natural next step is the application of the framework to granular policy level data within an insurance company, where the embeddings can be combined with exposure information and policyholder characteristics.

From an actuarial modelling perspective, spatial embeddings offer a flexible and scalable way to incorporate geographic information into pricing models. They overcome the limitations of entity embeddings and spatial smoothing methods and can be used in combination with other modelling techniques. The embeddings are smooth, transferable, and computationally efficient, making them suitable for large-scale applications in insurance analytics. A further advantage of the proposed framework is that it can be trained on modest computational hardware and easily adapted to new regions or additional spatial data sources, making it practical for in-house deployment at insurance companies and other organisations.

An important consideration for the deployment of spatial embeddings in pricing is fairness. The embeddings are more granular than traditional territorial variables such as postal codes and may therefore pick up socioeconomic patterns not captured at the postal code level alone. When used as rating factors, they could act as a proxy for characteristics that are protected or sensitive. Combining our framework with the literature on fairness in insurance pricing (Côté et al. 2025; Jamotton and Hainaut 2026) is therefore a relevant direction, both to quantify the extent to which spatial embeddings encode protected characteristics and to construct pricing models that use their spatial information while satisfying fairness requirements.

A limitation of our case studies is the temporal alignment between the outcome data and the spatial views, which were collected at the time of the study. The framework itself is not restricted in this respect, since the embedding model can be retrained per calendar year, or per any other relevant period, with temporally matched imagery and map data. Such data are available, for example, through the historical satellite archives in Google Earth Engine (Gorelick et al. 2017) and snapshots of OpenStreetMap reconstructed from edit timestamps. The reported training times

show that such periodic retraining is computationally feasible. Extending the framework with an explicit temporal dimension, so that a single model produces embeddings for any location and point in time, is a promising direction for future work.

This work represents a step toward integrating the principles of foundation models into actuarial science. By demonstrating how multimodal spatial pretraining produces reusable embeddings that enhance predictive accuracy and generalise across space, we show that actuaries can benefit from adopting foundation-model approaches. Similar to the transfer of language and vision models to countless downstream tasks, spatial foundation models can serve as general-purpose representations that can be adapted for pricing, reserving, portfolio analytics, and emerging climate-related risks. While recent global-scale spatial foundation models, such as AlphaEarth (Brown et al. 2025), illustrate what is possible at the planetary scale, our work highlights a complementary direction: lightweight, region-specific spatial embeddings that can be retrained quickly, incorporate organisation-specific spatial views, and capture fine-grained local structure that global models may overlook. Our work enables further exploration of such models, to evaluate how pretrained spatial representations can be transferred across products, regions, and lines of business, and to incorporate domain expertise in shaping foundation models that are both scientifically rigorous and practically relevant for risk management.

Acknowledgements

Katrien Antonio gratefully acknowledges funding from the FWO and Fonds De La Recherche Scientifique-FNRS (F.R.S.-FNRS) under the Excellence of Science (EOS) program, project ASTeR-ISK Research Foundation Flanders [grant number 40007517]. The authors gratefully acknowledge support from the FWO network W001021N. Christopher Blier-Wong acknowledges financial support from the Natural Sciences and Engineering Research Council of Canada (RGPIN-2025-06879). We also express our gratitude to the editor and reviewers for their valuable feedback, which has greatly enhanced the quality of this article.

Competing interests

The authors declare no potential conflicts of interest.

Supplemental material

The code to train the model architecture can be found via the GitHub repository: <https://github.com/freekholvoet/MultiviewSpatialEmbeddings>. The trained models, as defined in Table 1, are available in Holvoet (2025).

References

- Assuralia (2025). Klimaatschademonitor. <https://www.assuralia.be/nl/klimaatschademonitor>. Accessed March 2026.

- Avanzi, B., Taylor, G., Wang, M., and Wong, B. (2024). Machine learning with high-cardinality categorical features in actuarial applications. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 54(2):213–238. doi: [10.1017/asb.2024.7](https://doi.org/10.1017/asb.2024.7).
- Blier-Wong, C., Baillargeon, J.-T., Cossette, H., Lamontagne, L., and Marceau, E. (2020). Encoding neighbor information into geographical embeddings using convolutional neural networks. In *Proceedings of the Thirty-Third International FLAIRS Conference*. <https://aaai.org/papers/15-flairs-2020-18400/>.
- Blier-Wong, C., Baillargeon, J.-T., Cossette, H., Lamontagne, L., and Marceau, E. (2021). Rethinking representations in P&C actuarial science with deep neural networks. *arXiv preprint arXiv:2102.05784*. doi: [10.48550/arXiv.2102.05784](https://doi.org/10.48550/arXiv.2102.05784).
- Blier-Wong, C., Cossette, H., Lamontagne, L., and Marceau, E. (2022). Geographic ratemaking with spatial embeddings. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 52(1):1–31. doi: [10.1017/asb.2021.25](https://doi.org/10.1017/asb.2021.25).
- Boskov, M. and Verrall, R. (1994). Premium rating by geographic area using spatial models. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 24(1):131–143. doi: [10.2143/AST.24.1.2005085](https://doi.org/10.2143/AST.24.1.2005085).
- Brown, C. F., Kazmierski, M. R., Pasquarella, V. J., Rucklidge, W. J., Samsikova, M., Zhang, C., Shelhamer, E., Lahera, E., Wiles, O., Ilyushchenko, S., Gorelick, N., Zhang, L. L., Alj, S., Schechter, E., Askay, S., Guinan, O., Moore, R., Boukouvalas, A., and Kohli, P. (2025). AlphaEarth foundations: An embedding field model for accurate and efficient global mapping from sparse label data. *arXiv preprint arXiv:2507.22291*. doi: [10.48550/arXiv.2507.22291](https://doi.org/10.48550/arXiv.2507.22291).
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. doi: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- Côté, O., Côté, M.-P., and Charpentier, A. (2025). A fair price to pay: Exploiting causal graphs for fairness in insurance. *Journal of Risk and Insurance*, 92(1):33–75. doi: [10.1111/jori.12503](https://doi.org/10.1111/jori.12503).
- Delong, L. and Kozak, A. (2023). The use of autoencoders for training neural networks with mixed categorical and numerical features. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 53(2):213–232. doi: [10.1017/asb.2023.15](https://doi.org/10.1017/asb.2023.15).
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255. doi: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- Dimakos, X. K. and Di Rattalma, A. F. (2002). Bayesian premium rating with latent structure. *Scandinavian Actuarial Journal*, 2002(3):162–184. doi: [10.1080/034612302320179854](https://doi.org/10.1080/034612302320179854).
- Eilers, P. H. C. and Marx, B. D. (1996). Flexible smoothing with B-splines and penalties. *Statistical Science*, 11(2):89–121. doi: [10.1214/ss/1038425655](https://doi.org/10.1214/ss/1038425655).
- Etalab (n.d.). Demandes de valeurs foncières géolocalisées [Data set]. <https://www.data.gouv.fr/en/datasets/demandes-de-valeurs-foncieres-geolocalisees/>.
- Eurostat (2021). Local administrative units (LAU) [data set]. <https://ec.europa.eu/eurostat/web/nuts/local-administrative-units>. Accessed: 2024-08-28.
- Frees, E. W., Meyers, G., and Cummings, A. D. (2011). Summarizing insurance scores using a Gini index. *Journal of the American Statistical Association*, 106(495):1085–1098. doi: [10.1198/jasa.2011.tm10506](https://doi.org/10.1198/jasa.2011.tm10506).
- Frees, E. W., Meyers, G., and Cummings, A. D. (2014). Insurance ratemaking and a Gini index. *Journal of Risk and Insurance*, 81(2):335–366. doi: [10.1111/j.1539-6975.2012.01507.x](https://doi.org/10.1111/j.1539-6975.2012.01507.x).

- Gao, L. and Shi, P. (2022). Leveraging high-resolution weather information to predict hail damage claims: A spatial point process for replicated point patterns. *Insurance: Mathematics and Economics*, 107:161–179. doi: <https://doi.org/10.1016/j.insmatheco.2022.08.006>.
- Geerts, M., vanden Broucke, S., and De Weerd, J. (2024). GeoRF: a geospatial random forest. *Data Mining and Knowledge Discovery*, 38(6):3414–3448. doi: [10.1007/s10618-024-01046-7](https://doi.org/10.1007/s10618-024-01046-7).
- Google (n.d.). Google Maps Static API. <https://developers.google.com/maps/documentation/maps-static>. Accessed: 2024-10-28.
- Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., and Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202:18–27. doi: [10.1016/j.rse.2017.06.031](https://doi.org/10.1016/j.rse.2017.06.031).
- Gschlößl, S. and Czado, C. (2007). Spatial modelling of claim frequency and claim size in non-life insurance. *Scandinavian Actuarial Journal*, 2007(3):202–225. doi: [10.1080/03461230701414764](https://doi.org/10.1080/03461230701414764).
- Guo, C. and Berkhahn, F. (2016). Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*. doi: [10.48550/arXiv.1604.06737](https://doi.org/10.48550/arXiv.1604.06737).
- Hastie, T., Tibshirani, R., and Friedman, J. H. (2009). *The elements of statistical learning: data mining, inference, and prediction*. Springer Series in Statistics. Springer New York, New York, NY. doi: [10.1007/978-0-387-84858-7](https://doi.org/10.1007/978-0-387-84858-7).
- He, K., Fan, H., Wu, Y., Xie, S., and Girshick, R. (2020). Momentum contrast for unsupervised visual representation learning. In *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9726–9735. doi: [10.1109/CVPR42600.2020.00975](https://doi.org/10.1109/CVPR42600.2020.00975).
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. doi: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- Henckaerts, R., Antonio, K., Clijsters, M., and Verbelen, R. (2018). A data driven binning strategy for the construction of insurance tariff classes. *Scandinavian Actuarial Journal*, 2018(8):681–705. doi: [10.1080/03461238.2018.1429300](https://doi.org/10.1080/03461238.2018.1429300).
- Henckaerts, R., Côté, M.-P., Antonio, K., and Verbelen, R. (2021). Boosting insights in insurance tariff plans with tree-based machine learning methods. *North American Actuarial Journal*, 25(2):255–285. doi: [10.1080/10920277.2020.1745656](https://doi.org/10.1080/10920277.2020.1745656).
- Holvoet, F. (2025). Spatial embeddings via multiview contrastive learning. [Pretrained spatial embedding models]. doi: [10.57967/hf/7009](https://doi.org/10.57967/hf/7009).
- Holvoet, F., Antonio, K., and Henckaerts, R. (2025). Neural networks for insurance pricing with frequency and severity data: A benchmark study from data preprocessing to technical tariff. *North American Actuarial Journal*, 29(3):519–562. doi: [10.1080/10920277.2025.2451860](https://doi.org/10.1080/10920277.2025.2451860).
- Hoogeboom, E., Peters, J. W. T., Cohen, T. S., and Welling, M. (2018). HexaConv. In *Proceedings of the International Conference on Learning Representations*. <https://openreview.net/forum?id=r1vuQG-CW>.
- Jamotton, C. and Hainaut, D. (2026). A multivariate energy-based fairness adjuster for premiums. *North American Actuarial Journal*, pages 1–43. doi: [10.1080/10920277.2026.2705183](https://doi.org/10.1080/10920277.2026.2705183).
- Kingma, D. P. and Ba, J. L. (2015). Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*. doi: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980).
- Klemmer, K., Rolf, E., Robinson, C., Mackey, L., and Rußwurm, M. (2025). SatCLIP: Global, General-Purpose Location Embeddings with Satellite Imagery. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39(4), pages 4347–4355. doi: [10.1609/aaai.v39i4.32457](https://doi.org/10.1609/aaai.v39i4.32457).

- Klemmer, K., Rolf, E., Robinson, C., Mackey, L., and Rußwurm, M. (2023). Towards global, general-purpose pretrained geographic location encoders. In *Proceedings of the NeurIPS 2023 Workshop on Tackling Climate Change with Machine Learning*. <https://www.climatechange.ai/papers/neurips2023/114>.
- Lorenz, M. O. (1905). Methods of measuring the concentration of wealth. *Publications of the American Statistical Association*, 9(70):209–219. doi: 10.2307/2276207.
- Mai, G., Huang, W., Sun, J., Song, S., Mishra, D., Liu, N., Gao, S., Liu, T., Cong, G., Hu, Y., Cundy, C., Li, Z., Zhu, R., and Lao, N. (2024). On the opportunities and challenges of foundation models for GeoAI (vision paper). *ACM Transactions on Spatial Algorithms and Systems*, 10(2):1–46. doi: 10.1145/3653070.
- Mai, G., Janowicz, K., Hu, Y., Gao, S., Yan, B., Zhu, R., Cai, L., and Lao, N. (2022). A review of location encoding for GeoAI: Methods and applications. *International Journal of Geographical Information Science*, 36(4):639–673. doi: 10.1080/13658816.2021.2004602.
- Mai, G., Janowicz, K., Yan, B., Zhu, R., Cai, L., and Lao, N. (2020). Multi-scale representation learning for spatial feature distributions using grid cells. In *Proceedings of the International Conference on Learning Representations*. https://iclr.cc/virtual_2020/poster_rJljdh4KDH.html.
- Mai, G., Xie, Y., Jia, X., Lao, N., Rao, J., Zhu, Q., Liu, Z., Chiang, Y.-Y., and Jiao, J. (2025). Towards the next generation of geospatial artificial intelligence. *International Journal of Applied Earth Observation and Geoinformation*, 136: Article 104368. doi: 10.1016/j.jag.2025.104368.
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. doi: 10.48550/arXiv.1301.3781.
- Moussaid, S. (2023). Convolutional regional autoencoders for spatial embedding. Master’s thesis, KU Leuven.
- Olden, J. D., Joy, M. K., and Death, R. G. (2004). An accurate comparison of methods for quantifying variable importance in artificial neural networks using simulated data. *Ecological modelling*, 178(3-4):389–397. doi: 10.1016/j.ecolmodel.2004.03.013.
- OpenStreetMap contributors (n.d.). Openstreetmap [map data]. <https://www.openstreetmap.org>. Accessed: 2024-08-30.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>.
- Richman, R. (2021). AI in actuarial science – a review of recent advances – part 1. *Annals of Actuarial Science*, 15(2):207–229. doi: 10.1017/S1748499520000238.
- Richman, R. and Wüthrich, M. V. (2024). High-cardinality categorical covariates in network regressions. *Japanese Journal of Statistics and Data Science*, 7:921–965. doi: 10.1007/s42081-024-00243-4.
- Robben, J., Antonio, K., and Kleinow, T. (2026). The short-term association between environmental variables and mortality: evidence from europe. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 189(2):1131–1153. doi: 10.1093/jrsssa/qnaf052.
- Rußwurm, M., Klemmer, K., Rolf, E., Zbinden, R., and Tuia, D. (2024). Geographic location encoding with spherical harmonics and sinusoidal representation networks. In *Proceedings of the International Conference on Learning Representations*. <https://iclr.cc/virtual/2024/>

- [poster/18690](#).
- Shi, P. and Shi, K. (2017). Territorial risk classification using spatially dependent frequency-severity models. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 47(2):437–465. doi: [10.1017/asb.2017.7](#).
- Shi, P. and Shi, K. (2023). Non-life insurance risk classification using categorical embedding. *North American Actuarial Journal*, 27(3):579–601. doi: [10.1080/10920277.2022.2123361](#).
- Sitzmann, V., Martel, J., Bergman, A., Lindell, D., and Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pages 7462–7473. doi: [10.48550/arXiv.2006.09661](#).
- Snyder, W. E., Qi, H., and Sander, W. A. (1999). Coordinate system for hexagonal pixels. In *Proceedings of Medical Imaging 1999: Image Processing*, volume 3661, pages 716 – 727. doi: [10.1117/12.348629](#).
- Steppa, C. and Holch, T. L. (2019). HexagDLy: Processing hexagonally sampled data with CUDA-enabled convolutions in PyTorch. *SoftwareX*, 9:193–198. doi: [10.1016/j.softx.2019.02.010](#).
- Tarasiou, M., Chavez, E., and Zafeiriou, S. (2023). ViTs for SITS: Vision Transformers for Satellite Image Time Series. In *Proceedings of the 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10418–10428. IEEE. doi: [10.1109/CVPR52729.2023.01004](#).
- Taylor, G. (2001). Geographic premium rating by Whittaker spatial smoothing. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 31(1):147–160. doi: [10.2143/AST.31.1.999](#).
- Taylor, G. C. (1989). Use of spline functions for premium rating by geographic area. *ASTIN Bulletin: The Journal of the International Actuarial Association*, 19(1):91–122. doi: [10.2143/AST.19.1.2014917](#).
- Uber Technologies, Inc. (n.d.). H3: A hexagonal hierarchical spatial index [computer software]. <https://h3geo.org>. Accessed: 2024-08-30.
- Wang, Y., Braham, N. A. A., Xiong, Z., Liu, C., Albrecht, C. M., and Zhu, X. X. (2023). SSL4EO-S12: A large-scale multimodal, multitemporal dataset for self-supervised learning in earth observation [software and data sets]. *IEEE Geoscience and Remote Sensing Magazine*, 11(3):98–106. doi: [10.1109/MGRS.2023.3281651](#).
- Wilsens, P., Antonio, K., and Claeskens, G. (2025). Reducing the dimensionality and granularity in hierarchical categorical variables. *Advances in Data Analysis and Classification*, 19:1087–1118. doi: [10.1007/s11634-024-00614-5](#).
- Yin, Y., Liu, Z., Zhang, Y., Wang, S., Shah, R. R., and Zimmermann, R. (2019). GPS2Vec: Towards generating worldwide GPS embeddings. In *Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 416–419. doi: [10.1145/3347146.3359067](#).

A OpenStreetMap tags per category

Table 6 lists the OpenStreetMap tags used in each of the six categories as defined in Section 2. Each category lists the tag type, as defined by OpenStreetMap, along with the names of the tags collected per type.

Category	OpenStreetMap tag type	Tag names
shop	amenity	shops
tourism	amenity	tourism
food&drink	amenity	bar, cafe, pub, restaurant, fast food, food court, events venue, nightclub
health	amenity	clinic, dentist, doctors, hospital, pharmacy, social facility
	healthcare	clinic, doctor, hospital, pharmacy
education	amenity	college, kindergarten, research institute, school, university
public services	amenity	fire station, police

Table 6: The OpenStreetMap tags, with the tag type, for each of the six categories defined in Section 2.

B Hyperparameter selection

Hyperparameter selection for the spatial embedding model was carried out during model construction. As the embeddings are trained in a self-supervised manner, their quality cannot be directly evaluated using standard predictive metrics. Instead, the aim was to identify configurations that produced stable training behaviour and embeddings containing spatially informative structure.

A range of model configurations was tested for the OpenStreetMap encoder, location encoder, and fusion layer, varying the number of filters, neurons, and learning rates. Each configuration was pretrained for a fixed number of epochs and evaluated on two criteria. First, the contrastive loss (CLIP loss) was monitored to ensure stable convergence. Second, the resulting embeddings were applied in small-scale downstream tasks, such as logistic regression for country classification and a regression model for population density prediction. For both downstream tasks, we compared the performance of models with the embedding vector as input versus those with latitude and longitude coordinates. Configurations showing both stable loss behaviour and high downstream classification accuracy were retained.

The final hyperparameter values, reported in Section 3.4, combine established settings from Klemmer et al. (2025) with those identified through these preliminary tests. Smaller convolutional architectures (8 and 16 filters of size $k = 1$) and a 128-neuron fusion layer yielded consistent results, while larger models offered limited improvement and less stable optimisation. As in most self-supervised learning setups, the hyperparameter selection is therefore empirical, guided by training stability and representational quality rather than formal optimisation.

C Land use classification codes

Table 7 lists all land use codes in the dataset in Section 4.1.1. The original description in French is given, along with a translation.

Code	Original description	English translation
AB	terrains à bâtir	building plot
AG	terrains d’agrément	amenity land
B	bois	woodland
BF	futaies feuillues	deciduous forest
BM	futaies mixtes	mixed forest
BO	oseraies	willow grove
BP	peupleraies	poplar grove
BR	futaies résineuses	coniferous forest
BS	taillis sous futaie	coppice with standards
BT	taillis simples	simple coppice
CA	carrières	quarry
CH	chemin de fer	railway
E	eaux	water body
J	jardins	garden plot
L	landes	heathland
LB	landes boisées	wooded heath
P	prés	meadow
PA	pâtures	pasture
PC	pacages	grazing land
PE	prés d’embouche	grazing meadow
PH	herbages	grassland
PP	prés plantes	crop meadow
S	sols	residential plot
T	terres	arable land
TP	terres plantées	planted arable land
VE	vergers	orchard
VI	vignes	vineyard

Table 7: Land-use classification codes for the variable `code_nature_culture` in the dataset introduced in Section 4.1.1. The original French description is provided alongside the English translation.